altairsim — Developer Guide

2026-10-15 · f758c89

A simulation of the MITS Altair 8800 and the S-100 bus

Building it 1
Theory of operation 9
Writing a board 24
Writing a video board 36
Serial I/O 44
The built-in terminal 55
Soft-sector floppy controllers 60
Banked memory — the model, and where it was wrong 69
The PS II monitor loader, inside 73
Assembling a ROM (`.ASM` → `.HEX`) 76
Review comments 79

Building it

There are no dependencies. A C++20 compiler and CMake ≥ 3.20 is the entire list. The TOML parser, the JSON encoder and the line editor are all in the tree, so a fresh clone builds with nothing to download.

```
git clone https://github.com/deltecent/altairsim.git
cd altairsim
cmake -S . -B build && cmake --build build -j
ctest --test-dir build -LE slow      # drop -LE slow for the full 8080 exerciser
./build/altairsim                    # the default machine
```

If CMake is unfamiliar, the two `cmake` calls are two distinct steps — **configure**, then **build** — and you run both in that order:

- **`cmake -S . -B build`** — *configure*. Reads `CMakeLists.txt` in the source tree (`-S .`, the current directory) and writes the generated build files into a fresh `build/` directory (`-B build`). It downloads nothing and compiles nothing; it just works out *how* to build. You rerun it only when `CMakeLists.txt` itself changes — and even then the build step reruns it for you.
- **`cmake --build build -j`** — *build*. Compiles what the configure step laid out under `build/`. `-j` runs the compiles in parallel across all CPU cores (drop it for a serial build, or `-j4` to cap the jobs). This is the command you repeat after editing code.
- **`ctest --test-dir build -LE slow`** — run the tests that the build produced. `--test-dir build` points at the same directory; `-LE` is *label-exclude*, so `-LE slow` runs everything **except** tests tagged `slow` (the ~10-minute 8080 exerciser). Drop it to run those too.

Everything lands in `build/`, which is disposable: `rm -rf build` and rerun the configure step for a clean slate.

That is not a boast about minimalism for its own sake. It is a property worth defending: the day this needs a package manager to build is the day it stops being something a person can pick up in ten years and compile.

After a `git pull`, the commands do not change. `cmake -S . -B build && cmake --build build` again — there is no separate incremental procedure and no need to delete `build/`. New `.cpp` files can't be missed because sources are listed in `CMakeLists.txt` rather than globbed, so adding one edits `CMakeLists.txt` and forces a reconfigure; `roms/` and `machines/` *are* globbed but use `CONFIGURE_DEPENDS`, so they re-glob on every build. The build directory does cache absolute paths, though, so **rename the checkout or switch compilers and you must `rm -rf build`** — it fails with a loud `CMakeCache.txt directory is different` error, not a silent wrong answer. If a build breaks — or a test that used to pass fails — immediately after a pull, suspect a stale `build/` before suspecting the code: a `rm -rf build` and a clean reconfigure is the first thing to rule out, because a half-rebuilt object can walk an already-fixed bug back in. `docs/building-linux.md` §7 has the details.

Warnings as errors before a PR

The build always compiles with `-Wall -Wextra -Wpedantic` (GCC/Clang) or `/W4 /permissive-` (MSVC), but by default a warning is just noise and does not fail anything. **CI configures every leg with `-DWERROR=on`**, which on GCC and Clang adds `-Werror` and turns a warning into a build failure — so a PR that introduces one on either of those goes red before it can merge. Reproduce that gate locally before you open the PR:

```
cmake -B build -DWERROR=on && cmake --build build -j
```

It is off by default (`option(WERROR … OFF)` in `CMakeLists.txt`) so a casual build, or one on an unfamiliar toolchain whose newer diagnostics have not been chased down yet, still compiles. GCC and Clang have slightly different warning sets, so a clean local build on one is not proof the other is clean — that is what the separate CI legs exist to catch.

MSVC is not in the gate yet. `/W4` carries a pre-existing backlog (intentional integer narrowing, which GCC/Clang do not flag, plus some variable shadowing) that must be cleared before `/WX` can go on. That is tracked in issue #238; until it lands, `-DWERROR=on` is a no-op on MSVC and its `/W4` warnings stay visible in the log but non-fatal.

The optional video backend (SDL3)

Still no *required* dependency. The graphics boards — the VDM-1, and the Dazzler to follow — draw into a host `Display` (`src/host/display.h`). That interface has two backends: a headless `NullDisplay` (draws into memory, shows nothing) that is always available, and an `SdlDisplay` (a real window) compiled **only when SDL3 is found**. So a plain `cmake -S . -B build` on a machine with no SDL3 still configures, builds, and passes every test — the video boards are simply headless, which is exactly what CI and a scripted run want.

To get a window, install SDL3 and reconfigure:

```
# macOS
brew install sdl3
# Debian/Ubuntu (needs a recent SDL3 package)
sudo apt install libsdl3-dev
# vcpkg (any OS)
vcpkg install sdl3

cmake -S . -B build          # reconfigure -- watch the status line
```

The configure step reports which backend it chose:

```
-- SDL3 found -- video boards enabled (windowed)
        …or…
-- SDL3 not found -- video boards build headless (null display). Install SDL3 (see docs/devguide) for a
window.
```

CMake auto-detects with `find_package(SDL3 CONFIG)`; found → it compiles the `src/host/*_sdl.cpp` files, links `SDL3::SDL3`, and defines `ALTAIRSIM_ENABLE_SDL`. Force a headless build even where SDL3 is installed with **`-DALTAIRSIM_ENABLE_SDL=OFF`**. That flag is what a macOS *universal* build needs, because a Homebrew SDL3 is single-arch and cannot link into an `x86_64;arm64` fat binary. **CI passes it nowhere. One leg — macOS — installs SDL3 from Homebrew and builds native `arm64`, so it is the only place the `*_sdl.cpp` files are compiled at all**, and the workflow fails that leg if it comes up headless. Linux and Windows take the not-found path above and build against the null display. Only the `src/host/*_sdl.cpp` files (display, joystick, audio) and the composition root (`src/main.cpp`) are macro-gated; the boards themselves `#include` no SDL and compile in every configuration. Then:

```
altairsim vdm1              # a VDM-1 with a banner-drawing demo (roms/VDM1DEMO)
```

What actually got built

Built and tested on Linux, macOS, and Windows. The code is written to be portable — C++20, no dependencies, and every OS difference confined to `src/platform/` behind a header with zero conditionals — and that portability is now proven, not asserted: Linux (Ubuntu/GCC), macOS on Apple Silicon (and on Intel, built natively there rather than in CI), and Windows on MSVC all build and pass the suite. The Windows platform layer, once merely written, is field-proven.

CI runs the suite on every push — on as many platforms as the change needs. `tools/ci-changed-code.sh` sorts a change into one of three kinds, and `ci.yml` picks the legs:

Change	Pull request	Push to `master`
docs — `docs/`, `reference/`, `.claude/`, a top-level `.md`, any `README.md`	Linux	Linux, or nothing if the PR tested the same files
code — `src/boards`, `chips`, `cli`, `mcp`, `tests/`, the data the tests read, `tools/`	Linux	Linux, macOS, Windows
core — `src/core`, `cpu`, `isa`, `platform`, `host`, `config`, `util`, `main.cpp`, the build, CI, any path not listed	Linux, macOS, Windows	Linux, macOS, Windows — build only, no tests, if the PR tested the same files

A `CMakeLists.txt` change that only adds non-core sources, comments and `add_test` blocks — what a new board does — counts as code, not core. Run the script against any commit to predict a run: `bash`

`tools/ci-changed-code.sh <base>`.

Most PRs are code: a board, a command, a test. Those are portable C++ above the platform layer, so the PR builds them on Linux, and the run on `master` after the merge builds all three. That run keeps `master`'s compiler cache warm — a PR can only read its own cache and `master`'s — and it is where an MSVC- or Clang-only warning in a board appears: minutes after the merge, not at the release. **A red leg on `master` is fixed before the next change merges.** A core change builds all three before the merge, because that is where the platforms really differ.

When a core merge changes no files — its tree is the PR head that CI passed — the run on `master` builds all three platforms but does not run the tests or upload the binaries. The PR run already passed the suite on each platform with those files; the build is kept because it saves `master`'s compiler cache.

Each leg uploads the binary it built. To fetch them:

```
tools/fetch-ci-binaries.sh          # newest CI run on the current branch
tools/fetch-ci-binaries.sh 42       # ...on PR 42
```

It **waits** for the run if it is still going and refuses to download from a red one. A run that built only Linux leaves one file; the `master` run after a code merge leaves all three. A build-only run leaves none, and the script tells you to fetch from the PR. They land in `./artifacts` (git-ignored, replaced on every fetch) with the executable bit restored, since the artifact zip does not carry POSIX modes. Nothing in the build or the tests reads from there — these are CI's binaries, kept for running or handing to someone, not a build output of this tree.

The tests

```
ctest --test-dir build -LE slow -j 8   # unit + acceptance, in parallel. Under a minute.
ctest --test-dir build              # ...plus 8080EXM, the full exerciser.
ctest --test-dir build -L hw        # a real null-modem cable, and a real TNFS server.
```

The acceptance tests are not unit tests. They **boot period software on the whole machine through the real CLI** and check what lands on the terminal — 4K and 8K BASIC off a cassette, MITS Programming System II polled and interrupt-driven, CP/M off a minidisk. Several ship with a **negative control**: the same script against a machine that ought to *fail*, marked `WILL_FAIL`. If a control ever passes, the test it guards was passing for the wrong reason and is worthless. That is the only reason to believe any of them.

The CPU exercisers are gated three ways, on purpose. `cpu-8080exm` is ~2.9 billion instructions and `cpu-zexdoc` / `cpu-zexall` ~5.8 billion each, so they are labelled `slow` and excluded from the per-push matrix. `cpu-exerciser.yml` runs them on one Linux runner *only when CPU/ISA code changes* — an opcode bug is a logic bug and shows up on any host, so one architecture is enough to catch it, and paying for three would be paying three times for the same answer. **A release then runs them on all four build**

machines (`tools/release-worker.sh`, `ctest -L slow`), through the binaries that ship, because the one thing that is *not* architecture-independent is the compiler: the cores are full of shifts, masks and half-carry arithmetic, and until a release nothing has ever driven billions of instructions through the MSVC build. `cpu-exerciser-release.yml` runs them on the three CI platforms on demand only.

The hardware tests (`-L hw`) run against an actual null-modem cable between two USB serial ports, because a claim about a cable deserves a cable. They are opt-in, pointed at your ports with `ALTAIR_SERIAL_A` / `ALTAIR_SERIAL_B`, and they **skip loudly** when the hardware is absent — a hardware test that quietly passes with no hardware is a green tick that means nothing. Unset, `serial-hw` exits 77 and ctest reports it `Skipped`; that is the expected result on any machine without the cable, not a failure.

The two device paths are per-machine — list yours, then point the test at two ports joined by the cable:

```
# macOS:   ls /dev/cu.usbserial-* /dev/tty.usbserial-*
# Linux:   ls /dev/ttyUSB* /dev/ttyACM*
# Windows: Device Manager -> Ports (COM & LPT), or run  mode

ALTAIR_SERIAL_A=/dev/ttyUSB0 ALTAIR_SERIAL_B=/dev/ttyUSB1 ctest --test-dir build -L hw
```

The cable is not just any null-modem cable. `serial-hw` drives the modem-control lines, so the pins have to actually cross — and it needs the *full-handshake* wiring in which **one end's DTR fans out to both DSR and DCD** on the other. A 3-wire cable (TxD/RxD/GND) carries the bytes but leaves every control pin dead; a partial null-modem that loops DTR/DSR/DCD back locally instead of crossing them looks identical to the eye and fails the same checks. That is the usual cause of a `serial-hw` that fails rather than skips — the byte check passes and the pin checks do not.

Wire it as (a DE-9 DTE pinout in parentheses, both ends):

A end		B end
TxD (3)	→	RxD (2)
RxD (2)	←	TxD (3)
RTS (7)	→	CTS (8)
CTS (8)	←	RTS (7)
DTR (4)	→	DSR (6) **and** DCD (1)
DSR (6) **and** DCD (1)	←	DTR (4)
GND (5)	↔	GND (5)

So B raising DTR is a *carrier appearing* at A — to a 6850 strapped `dcd=wired`, indistinguishable from a modem, which is the whole point of the test. The wiring is restated at the top of `tests/serialtest.cpp`.

`tnfs-hw` needs a TNFS server instead of a cable. It boots the CP/M example with its floppy mounted from a real `de-tnfsd` (a POSIX TNFS server, not part of this tree), `SAVE`s a file, and reads it back through a fresh mount. The script starts and stops the server itself, on a local port, serving a copy of the image. It runs when `de-tnfsd` is on `PATH` and skips (77) otherwise. On Windows it is not registered at all: `de-tnfsd` is POSIX-only. `tests/test_tnfs.cpp`, in `unit`, covers the same client against a fake server on every push.

Catching memory bugs: `-DSANITIZE=on`

A crash on one platform only — typically Windows CI red, macOS and Linux green — is usually a use-after-free or use-after-scope that one allocator tolerates and another does not. No compiler warning fires for it.

`-DSANITIZE=on` finds it locally. It threads AddressSanitizer + UndefinedBehaviorSanitizer (just ASan on MSVC) through the whole binary, so the crash names its own `file:line` instead of you guessing from a block-buffered stack trace. Configure it in its **own build directory** — an instrumented binary is slower and is not what you ship:

```
cmake -S . -B build-asan -DSANITIZE=on
cmake --build build-asan --target altair_tests
ASAN_OPTIONS=detect_leaks=0 UBSAN_OPTIONS=print_stacktrace=1 ./build-asan/altair_tests
```

It is off by default and not in CI — CI catches a platform-only crash by running on that platform, and this is the tool you reach for once it does.

The documentation is part of the build

Two documents come out of this tree:

Target	Is
User Manual (`docs/manual/`)	Ships in the package. **Self-contained** — it may not name a single file the reader does not have. No `src/`, no CMake, no `DESIGN.md`.
Developer Guide (`docs/devguide/`)	This. Repo only. Free to talk about the source, because you cannot write a board without it.

Some of the manual is **generated from the binary**, and this matters if you touch a board:

```
cmake --build build --target docs-reference
```

That rewrites `docs/manual/ref/*.md` — the command dictionary, the per-board parameter tables, the machine list — by walking `Board::properties()` and the `CommandDef` table. Those files are committed, and **a test (`docs-reference`) fails if they are stale.** So if you add a property, change a default, or reword a `HELP` string, run it and commit the result alongside your change.

This is not a docs chore bolted on at the end. A board's properties **are** its TOML schema; a hand-written parameter table would be a second schema, and the first draft of one in this very project got three of the memory board's eight defaults wrong. The reference is printed rather than retyped for the same reason the rest of the program has one source of truth for anything.

The PDFs need `pandoc` and any Chromium-based browser, and are built by `tools/build-docs.sh`. Neither tool is a build dependency, and neither goes anywhere near the simulator.

Why a wrapped paragraph pastes as several lines (and why we don't "fix" it)

A recurring report (issue #246): copy a paragraph out of one of our PDFs and it pastes as several lines instead of one. This is a **reader** behavior, and the investigation is worth recording so it is not re-run from scratch.

There are no hard line breaks in the paragraphs. In the PDF content stream each wrapped line is painted at a new baseline position; there is no newline character anywhere. When a paragraph pastes as three lines, the reader is *inventing* those breaks from the vertical gaps at copy time. Nothing in the build put them there.

The only thing that tells a reader "the real text of this span is *this exact string*, ignore the geometry" is `ActualText`. Hand-built PDFs extracted with poppler make the picture exact:

PDF	Copies as
Wrapped paragraph, plain geometry	2 lines (break invented)
Wrapped paragraph inside a real `<P>` **structure tag**	still 2 lines
Wrapped paragraph with paragraph-spanning **`ActualText`**	**1 line**

The middle row is the one that closes off the easy fixes: the **structure/accessibility tree does not join wrapped lines** — only `ActualText` does. Our PDFs are already tagged (`pdfinfo` → `Tagged: yes`), and Chrome emits `ActualText` **only for ligatures** (the `fi`/`fl` spans), never for paragraphs. Every mainstream HTML/Markdown→PDF engine (WeasyPrint, Prince, wkhtmltopdf, Typst, LaTeX) conveys paragraphs through the structure tree, not through paragraph-level `ActualText` — so **swapping the PDF engine changes the producer name and nothing about the copy behavior.** Do not propose one as the fix.

Readers split on this: Word and Acrobat honor the structure tree, so the paragraph comes across whole; Preview and Chrome's built-in viewer fall back to pure geometry and invent the breaks. Worse, **macOS Sequoia's Preview/PDFKit regressed** — it stamps a paragraph break on *every* visual line regardless of the document, surviving even a text-only paste — so it would ignore `ActualText` too.

A custom `ActualText` post-pass *is* buildable — Chrome already brackets each paragraph in one marked-content span — but it is a real tool, not a shell snippet: the paragraph text is emitted as subsetted **hex glyph-IDs**, so each glyph has to be reverse-mapped through the font's `ToUnicode` CMap, with TJ kerning arrays and nested ligature/italic spans handled, then the stream recompressed. For a cosmetic gain that the reader in the original report (Sequoia Preview) would not even honor, it has not been worth doing.

The Markdown can't fix it either: wrapping is decided at render time from the column width, and there is no "soft wrap that copies as one line" a source file can encode. The only source-side lever is keeping the things people copy short enough **not to wrap** — which the commands already are (mono, ≤79 columns, in a column sized for 79). The case that wraps is prose. For copy-a-command / copy-a-prompt workflows, copy from the **Markdown**, which ships beside every PDF and is plain text; the PDF is the read-it version.

Theory of operation

This chapter is about what is actually going on inside the machine. It assumes you have the source, and it names files: `src/core/bus.h`, `src/core/board.h`, `src/core/value.h`, `src/core/machine.h`, and `DESIGN.md`, which is the document all of this was argued out in.

The next chapter builds a board — a lamp latch on port `FF`. Everything here is what that board is standing on.

The whole architecture, in one sentence

> **Boards respond to bus cycles. The CPU originates them.**

That is not a slogan. It is the shape of the code, and everything else in this chapter is a consequence of it.

The two concepts are separate types, and they live together in `src/core/bus.h` rather than with the CPU:

```
class BusMaster {
public:
    virtual StepResult step(Bus&) = 0;
};
```

A CPU board is **both**: `Cpu8080Board : public Board, public BusMaster`. It is a card you can pull out with your hand (`BOARDS REMOVE cpu0` works, and a machine with no processor in it is a real machine — it is the one the monitor drove before the 8080 existed), and it is also the thing that drives cycles onto the backplane.

The payoff is DMA, and it is free. S-100 has `pHOLD` / `pHLDA` precisely because a backplane can have more than one master. A DMA board — a disk controller stealing cycles, a Dazzler — is a `Board` that *becomes* a `BusMaster` when it is granted the bus, and it drives the very same cycles through the very same interface the CPU uses. **DMA is never a special path bolted onto the bus.** If you find yourself writing one, the model has already gone wrong.

It is also why `BREAK MEM W 0100` catches a DMA write. A cycle is a cycle; nothing on the backplane records who originated it. There is deliberately **no `origin` field** on `BusCycle`, and `src/core/bus.h` says why: a real backplane cycle carries no such tag, which is exactly why a front-panel `DEPOSIT` is indistinguishable from a CPU write, and why a real ROM ignores both.

The bus carries signals. It does not invent behavior.

The second rule, and the one the bus header exists to enforce:

> The bus arbitrates no overlay, vectors no interrupt, knows no bank, and has never heard of ROM.

It picks no winner between two cards. It does not hand the CPU an interrupt vector. It does not route reads to one board and writes to another. Every one of those lives in a board.

When you are tempted to add `if (board is a ROM)` to `Bus`, **you have found a bug in your board instead.** There is exactly one thing the bus does that no board does, and §4.6.1 of DESIGN.md pins it down: it supplies `0xFF` when nobody answered. That is the entire bus/board overlap, and it stays that size.

> **This is the Altair bus, not IEEE-696.** The `BusCycle` above carries an 8080 status `Cycle`, a 16-bit address, and separate in/out data — the 1975 bus, not the 1983 standard that folded the data lines to 16 bits, stretched the address to 24, and grounded the front-panel PROTECT / sense-switch pins this simulator's `fp` board relies on. That is a deliberate line, not a gap to close later; **DESIGN.md §4.0** argues it out, including why an Altair board and a 696 board cannot honestly share one backplane.

A bus cycle, end to end

```
enum class Cycle { MemRead, MemWrite, IoRead, IoWrite, IntAck };

struct BusCycle {
    Cycle type = Cycle::MemRead;
    uint16_t addr = 0;   // memory address; for I/O the port is addr & 0xFF
    uint8_t data = 0;
    bool phantom = false;

    uint8_t port() const { return (uint8_t)(addr & 0xFF); }
    bool isWrite() const { return type == Cycle::MemWrite || type == Cycle::IoWrite; }
};
```

Five cycle types. `IntAck` is one of them, and that is load-bearing — see interrupts, below.

`data` is valid on a write. **On a read it is zero while the cycle is in flight** — nobody has driven the bus yet when `decodes()` and `read()` are asked, so there is nothing honest to put there — and it is **back-filled** with the byte that came back (a board's, or the floating bus's `0xFF`) before `snoop()` and the observers see it. `Bus::settle()`.

The bus runs each cycle in three passes:

Pass	What it does
1	Ask every board whether it pulls `PHANTOM*` → set `BusCycle::phantom`
2	Ask who `decodes()` this cycle. Exactly one should answer. Nobody → `0xFF`.
3	`settle()`: show the completed cycle to every board that asked to see it.

Carrying a signal, running a decode, and letting the cards watch. **No decisions.**

The decode is cached, because on real hardware it is wired

A card's address decoder is combinational logic — a PAL, a row of gates — wired to the address lines and to the status strobes `sMEMR`, `sINP`, `sOUT`. It does not *answer a question* once per cycle. It **settles**, and it only changes when something latches: a bank strap, `PHANTOM*`, a card pulled from the backplane.

So the bus asks the same questions of the same boards in slot order, and asks them **once**, storing the answer in four tables:

```
struct Slot {
    Board* who = nullptr;   // the single board that drives. null: nobody -> floats 0xFF
    bool phantom = false;   // PHANTOM* as resolved for this page and this cycle class
    bool slow = false;      // more than one driver. Contention: take the exact path.
};

Slot memRead_[256], memWrite_[256];
Slot ioRead_[256], ioWrite_[256];
```

One entry per **256-byte page** for memory, one per **port** for I/O, per cycle class. A decode is now a table lookup, not a walk over every board in the machine.

The board still owns the entire decision. The bus still invents nothing. It just stopped asking sixty-five thousand times a second — and stopped asking cards questions they are not even wired for. An I/O-only 2SIO used to be asked to decode every memory read, and its first act was to throw the question away. **A real 2SIO has no connection to the memory read strobe. It is not in that conversation.**

Three consequences follow, and all three are things you can get wrong.

`decodes()` must be pure

```
virtual bool assertsPhantom(const BusCycle&) const { return false; }
virtual bool decodes(const BusCycle&) const { return false; }
```

Same board state, same cycle → same answer, and no side effects. Both of these are combinational. The bus calls them several times per cycle — once to resolve the signal, again inside each board's own decode — and then it *caches the result*.

A `decodes()` with a side effect in it is a bug that will not surface for a month. It will fire the right number of times on the day you write it and the wrong number of times forever after, because the number of times the bus asks is an implementation detail of a cache. Latch nothing here. The clocked half is `snoop()`.

If your decode changes, say so

```
void decodeChanged();   // "my decode just changed" -- sets a dirty flag on the bus
```

A bank strap moved. A `PHANTOM*` jumper moved. A chip came out of a socket. The board went `enabled = false`. **Call `decodeChanged()`.** Forget, and the tables go stale and the machine lies quietly, which is the worst failure mode there is.

You mostly get this for free: `setProperty()` — the one path by which any property is *ever* set, from `SET`, from TOML, from `BOARDS ADD`, from MCP — calls `configChanged()` on the board after every successful set, and the default `configChanged()` calls `decodeChanged()` and `intChanged()`. You call it by hand for changes that do not come through a property: a guest `OUT` that moves a bank strap, a boot ROM switching itself out.

And it is not left to trust. `Bus::setVerify(true)` re-derives the decode the slow way on every single cycle and screams the moment it disagrees with the table. The unit suites run with it on permanently; the 8080 validation gate runs with it over 2.9 billion instructions. **It is a proof, not a path** — it is slower than the code it replaced, and that is fine.

The bus routes by CYCLE TYPE, not just by address

Four tables, not one. That is not an optimization detail — it is the model:

A card is wired to `sMEMR`, `sINP` and `sOUT` separately. A ROM famously does not decode a write *at all*; it does not reject the write, or ignore it, or log it — it never answers the cycle. So "who answers here" has a **different answer for a read than for a write**, and that falls out of the model rather than being bolted onto it.

The consequence you will use immediately: **one board can own `IN FF` while a completely different board owns `OUT FF`, with no contention whatsoever.** That is not a trick. It is what the Altair front panel actually does. `src/boards/mits-frontpanel.h` decodes exactly this and nothing else:

```
bool decodes(const BusCycle& c) const override {
    return enabled_ && c.type == Cycle::IoRead && c.port() == 0xFF;
}
```

Its sense-switch buffers are gated with `sINP`. There is no `sOUT` anywhere near them, so an `OUT 0FFH` **is not the panel's** — it goes unclaimed and the backplane discards the byte, which is precisely what the hardware does with it. Port `FF` output is therefore a free space on a real Altair, and it is where the next chapter's lamp board lives.

`decodeIsPageUniform()` — when a card decodes a low address line

```
virtual bool decodeIsPageUniform() const { return true; }
```

Memory decode is cached one entry per 256-byte page, and that is a **contract on `decodes()`**. Nearly every card can keep it: S-100 memory decoding comes off the *high* address lines, and a card selected at 1K or 4K granularity answers a whole page or none of it.

A card that decodes a low address line says `false`. The Tarbell is the real one — its PROM and its `PHANTOM*` are gated by **A5**, so it decodes `0000-001F` and *not* `0020-003F`: two different answers inside page 0. Say `false` and the bus probes every address of every page you might be in, and any page whose answer is not uniform is served by the exact, uncached two-pass path. **You lose nothing but the cache, and only on the pages you actually touch.**

`snoop()` — the clocked half, and the only place you may latch

```
virtual bool wantsSnoop() const { return false; }
virtual void snoop(const BusCycle&) {}
```

Every board sees every cycle. That is what a backplane IS — the address bus is not addressed *to* anyone, it is simply present, and any card may watch it whether or not it answers.

`snoop()` is called exactly **once per cycle, after it completes**, with `data` back-filled. It is the **only** place a board may latch what it saw. It is opt-in via `wantsSnoop()`, because calling a do-nothing virtual on every board on every cycle was pure ceremony.

The front panel says yes, and it is the clearest example of why the hook exists: **its lamps are wired to the backplane.** Not a metaphor — an LED on a bus line sees exactly the cycles `snoop()` hands you, and `FrontPanelBoard::snoop()` is the only place `addrLeds_`, `dataLeds_` and `status_` are written. The

lamps show whatever went by, and at 2 MHz that is a blur. It was a blur in 1975; the MITS manual says so.

The bus is not *notifying* anyone here. The cycle was on the backplane the whole time and they could all see it. This is only where we let them latch it.

`peek()` — look without touching

```
virtual bool peek(uint16_t addr, uint8_t& out) const { return false; }
```

A `read()` may CONSUME. An `IN` from a UART's data port takes the byte and the guest never sees it again. So `DISASM`, `TRACE` and the debugger's register display are built on `peek()`, never on a read — a disassembler that ate the console's input the first time someone disassembled a page with a UART mapped into it would be a debugger you could not trust.

`peek()` runs the same decode, `PHANTOM*` and all, so a shadowed board is invisible to it exactly as it is to a real read. It just never strobes anybody.

A board that cannot answer without side effects returns `false`, **and that is an honest answer, not a failure**: the byte on a real bus is only defined *during* a cycle. The caller shows `FF`, which is what the bus would have floated to.

The floating bus — one rule, three consequences

> **If no board drives the bus, it floats high. Every read of an unmapped address or port returns `0xFF`. Writes go nowhere.**

One rule. Three things that would otherwise each need a special case fall straight out of it:

Situation	What happens	Why it matters
Unpopulated memory	reads `0xFF`	Period software **sizes memory by reading**. Return `0x00` and every machine looks like it has 64K, and CP/M builds itself wrong.
Unmapped I/O port	reads `0xFF`	A guest probing for a board that is not there gets the answer real hardware gives it.
`IntAck` with nobody driving	reads `0xFF` = `RST 7`	The famous one. See below.

With no vector card in the machine: a board pulls pin 73, the CPU runs an `IntAck` cycle, nobody claims it, the bus floats, the CPU reads `FF` — and `FF` is `RST 7`. That is not a fallback anybody coded.

It is what a real Altair does, and it is why the PMMI's factory jumper straight to pin 73 yields `RST 7` with no vector logic anywhere in the machine.

Model the floating bus honestly once and all three are free. Fake any one of them and you will fake the other two differently.

Therefore no board may ever manufacture `0xFF`, and in particular no board may seed its own store with it. A RAM chip does not power up holding `FF`; it powers up holding whatever it feels like, which is what `fill = random` is for. Seed a board's store with `FF` and `DUMP` shows `FF` for a board whose RAM is fine, `FF` for a board whose RAM was never filled, and `FF` for a board that **isn't in the machine**. One symptom, three causes. The moment a board can produce `FF`, the only signal the bus has stops being a signal. `tests/test_boundary.cpp` enforces it.

Interrupts

Two wires' worth of interface, and the same rules apply to both.

```
virtual bool    assertsInt() const { return false; }   // pINT, S-100 pin 73
virtual uint8_t assertsVi()  const { return 0; }       // VI0-VI7, pins 4-11, as a BITMASK
virtual bool    watchesVi()  const { return false; }   // an 88-VI says yes; nothing else does
virtual int     intWinner()  const { return -1; }      // ...and only it has an opinion
void            intChanged();                          // "my pin may have moved"
```

An interrupt is a LEVEL, not an event

A UART with a character waiting and its interrupt jumper installed says `true`, **and keeps saying `true` until the guest reads the character.** There is no queue, so a board cannot "lose" an interrupt — there was never a queue to lose it from.

Both of these are **combinational and pure**, exactly like `decodes()`. `assertsInt()` reports the settled state of a pin, computed from the state of the chip and nothing else. It does not advance a receiver, take a byte off a line, or do any work the guest has not paid for.

It used to do all three, and there was a whole section of DESIGN.md defending it. The 6850 has to notice a character has finished arriving, which happens on the chip's own clock with no help from the CPU — and being asked `assertsInt()` was the only thing that ever woke the card up. **The poll was serving as the card's clock.** That work moved to where it belongs: a `Clock` deadline the card sets for itself, and `pump()`. Read §4.4.1 of DESIGN.md before you are tempted to do work inside one of these.

`assertsVi()` is a bitmask, not a level

Bit *n* means "I am pulling VI*n*". It is a mask and not a level because **a card can pull two lines at once**: the 88-SIO straps its input device and its output device independently, and the manual is explicit that they may sit at different priorities. Both can be asking in the same instant — a character has arrived *and* the transmitter has gone empty. A single `int` level would have to pick one and drop the other, silently, and only when both fired. That is the worst possible way to lose an interrupt.

Where a card's request is soldered is a **bus strap**, spelled the same on every card in the machine (`src/core/board.h`):

```
enum class IrqJumper { None, Int, Vi0, Vi1, Vi2, Vi3, Vi4, Vi5, Vi6, Vi7 };
Property irqJumperProperty(std::string name, std::string help, IrqJumper& j);
```

Use `irqJumperProperty()` and your board gets the same ten choices, the same spelling, the same tab completion, and a `SHOW BUS IRQ` line — and a card with *two* straps gets them both for free.

The bus does not poll. It keeps the wire.

`Bus::intPending()` used to walk the backplane and ask every card `assertsInt()`, **once per instruction** — sixty million times a second, to compute a boolean that changes maybe a thousand times a second. It was the single largest per-instruction cost left in the simulator once the decode was cached, and it *grew with every card you added*.

Now the board **pulls the pin** and the bus keeps a running wire-OR: `intCount_` for pin 73, and `viCount_[8]` plus a `viMask_` for the eight VI lines. Reading pin 73 is one integer test, flat in the number of cards.

The two hard-won rules turn out to be one rule:

	combinational, pure	**"it moved"**	**what the bus keeps**
address decode	`decodes()`	`decodeChanged()`	a page table
interrupt	`assertsInt()`	`intChanged()`	a wire-OR count

> **A board's outputs are pure functions of its state. When its state changes, it says so. The bus caches the rest.**

Call `intChanged()` after anything that could move the pin: a register written, a character taken off the line, a deadline coming due, a jumper moved. A spurious call costs a virtual call. **A missing one hangs the**

guest forever, waiting for an interrupt that already happened — and it presents as "*the emulator locks up sometimes*", which is worth a week of anyone's life.

So it is not left to trust either. `Bus::setVerify(true)` re-derives the **whole** wire — pin 73 and all eight VI lines — from every board's `assertsInt()` / `assertsVi()` on every instruction, and aborts the moment a board disagrees with the cache.

One call covers all nine wires on purpose. A board does not know which wire its jumper is in today, and a card that had to announce each wire separately would eventually forget one.

Where the vector comes from

Nowhere in the bus. **The vector comes from whoever claims the `IntAck` cycle — like any other cycle.**

That is the whole design. `watchesVi()` and `intWinner()` are the 88-VI's, and nothing else's: it watches the eight lines, applies its own priority and its own mask, drives pin 73 itself, and then claims `IntAck` and jams an `RST n` onto the data bus. It is an **ordinary board**. It has no special privileges, and neither will any new interrupt controller you invent — which is the point of the project.

`watchesVi()` exists because a card that *pulls* a VI line announces it, but the card *watching* those lines is a third party who was told nothing and whose own pin 73 has just gone stale. The bus calls `intChanged()` on each watcher when a VI line actually moves. `intWinner()` exists so `SHOW BUS IRQ` can say *which* line wins without the monitor knowing what an 88-VI is — the alternative was a `dynamic_cast` in the monitor, and **the monitor does not get to know about cards.**

Memory, ROM, and `PHANTOM*`

A memory card is **a list of regions**, not an address range: RAM here, a ROM socket there, an empty socket between them. One card can occupy two separate ranges, because a real one does. `SHOW BUS MAP` is per-*range*, not per-board, for that reason. It is also the bus as it is *now*: the monitor takes each range from the board's `memMap()` and asks `Bus::respondersTo()` which of its addresses the board answers, for a read or for a write. A PROM that the guest has switched out has no row. `memMap()` itself is a fixed description of the board, and `SHOW MACHINE` prints it unchanged.

An empty socket decodes nothing. It does not read as zero — it floats to `FF`, like anything else nobody drives.

`PHANTOM*` is a signal a board pulls

`PHANTOM*` is **S-100 pin 67**, a line any board may pull low.

```
virtual bool assertsPhantom(const BusCycle&) const { return false; }
```

A boot ROM pulls it. A memory board **strapped to honour it** takes *itself* off the bus for that cycle — it returns `false` from its own `decodes()`. **The bus picks no winner.** The overlay is *emergent*: the ROM is the only board still answering, because the RAM switched itself off.

That is the entire overlay mechanism, and it is how a boot PROM shadows RAM at `FF00` and then gets out of the way (`setEnabled(false)`, usually as its last act, usually triggered by the boot code writing to the ROM card's own port — an ordinary `OUT` the card decodes).

Two things fall out of it that you must not re-derive by hand:

- **A shadow is not contention.** `Bus::respondersTo()` returns every board that *actually* decodes, so a phantomed-out board does not appear in it, and `SHOW BUS CONTENTION` stays quiet. Contention is decided *after* the phantom pass, on who is really driving. Two cards *registered* at one address is normal and correct — that is what a boot ROM over RAM **is**.
- **A card must not switch itself off with a signal it is itself driving.** The memory board's decode carries a `!assertsPhantom(c)` clause for exactly that, and its absence was a real bug in the default machine: the ROM pulls the pin, honours the pin, disappears, and reads back `FF` off its own floating bus.

`PHANTOM*` is a LEVEL, and the asserting card has no opinion about writes. **The read/write distinction lives on the *honouring* board** — which is why the memory card's jumper is `honors_phantom = none | read | all` and not a `bool`. `read` means *stop answering reads, keep answering writes*, so the byte lands in the RAM under the shadow. That is what lets a Tarbell bootstrap write the sector it is loading into the very RAM its own PROM is covering. (This paragraph used to say the exact opposite, in bold, and it was wrong — reasoned instead of sourced. Patrick read the schematic.)

`rawRead` / `rawWrite` / `rawSize` — the PROM burner

```
virtual size_t  rawSize() const { return 0; }
virtual uint8_t rawRead(size_t) const { return 0xFF; }
virtual bool    rawWrite(size_t, uint8_t) { return false; }
```

Straight into the card's backing store, **bypassing decode entirely**. Offsets are board-local, and the store may be far larger than 64K — a banked card's bank 3 simply *is* offset `0x30000`.

This is the PROM burner, and that is not a metaphor. It is how the operator writes a ROM the guest cannot, because **burning a PROM is not a bus operation on real hardware either. You pull the chip.**

Model it as a bus write instead and the bus would have to know *who originated a cycle* — which a real backplane cannot know, and which no board should ever have to ask.

Reflection is the keystone

```
virtual std::vector<Property> properties() = 0;

struct Property {
    std::string name;                   // "baud", "phantom", "honors_phantom"
    std::string help;                   // one line, shown by SHOW
    Kind kind = Kind::Int;              // Int | Bool | Str | Enum

    std::vector<std::string> choices;   // Kind::Enum -- also feeds tab completion
    long long min = 0, max = 0;         // Kind::Int; min==max means unbounded
    int radix = 10;                     // 16 for addresses, so SHOW reads right
    std::string unit;                   // "Hz", "bytes" -- display only
    bool irqJumper = false;

    std::function<Value()> get;
    std::function<bool(const Value&, std::string& err)> set;
};
```

`SET`, `SHOW`, the TOML loader, `CONFIG SAVE`, the MCP tool schemas, tab completion **and the manual's generated board reference** are all written **once**, against this. They know nothing board-specific.

There is no second schema anywhere. A board's TOML keys *are* its properties. A board added next year is configurable, scriptable, agent-drivable and documented **the day it lands**, and none of those six consumers changes a line.

The cost of that is that a property has to carry enough metadata to validate, render and describe itself. That is the whole trick, and it is why `radix` is there: `PORT=10` is port `0x10` and `BAUD=9600` is nine thousand six hundred, because the property said so.

Three things to get right when you write one.

A property with no setter is read-only. Live pin state, a derived value, a lamp. Leave `set` empty — **and that absence is the only signal any consumer has.** A setter that always refuses would stop a `SET`, and simultaneously fool `SHOW`, `CONFIG SAVE`, the MCP schema and the generated docs, all four of which read the *presence* of the function. Do not write one.

There is no config-time-only property. Every property can be set, always. You can only type at the prompt when the machine is stopped — that is the front panel's STOP switch, and there is no moment at which a `SET` races a running CPU. And on real hardware the gate would be a fiction anyway: a card being worked on sits on an **extender**, out where you can reach it, and its jumpers get moved with the power on.

`unitProperties()` is for a real sub-thing with its own settings.

```
virtual std::vector<Property> unitProperties(const std::string& unit) { return {}; }
```

The two halves of a 2SIO are **two independent 6850s** with their own crystals' worth of jumpers — independent baud rates, independent transforms — so they are units, and `SET sio0:a BAUD=9600` reaches one of them. Folding them into `properties()` as `a_baud` / `b_baud` would work for a 2SIO and fall apart on the first card with eight ports. Most boards return `{}` here, and that is fine.

A unit is a **name and a kind** (`UnitDef`, `UnitKind::{Disk, Rom, Serial, Tape, Cpu}`), never an index — so `MOUNT dj:drive0`, and mounting a disk image onto a serial port is an *error with a sentence*, not undefined behaviour that half-works. `units()` is the only list; `SHOW`, `MOUNT`, `CONNECT`, the MCP schemas and tab completion all read it, so they cannot disagree about what exists.

Reset is two different events

```
enum class Reset {
    PowerOn,  // POC* -- pin 76. Nothing in software can assert it; only the power supply.
    Bus       // RESET* -- the front-panel button. Warm.
};

virtual void reset(Reset) {}
virtual void power() {}
```

Mixing these up is the classic source of "*works from power-on but not from the reset button.*"

Neither reset clears memory. Only removing power does. That is not a nuance, it is the rule, and the memory array is the proof: **a RAM chip has no POC* pin.** Its contents are indeterminate at power-up because *the chips just powered up*, not because a signal arrived. A `Reset::Bus` leaves memory intact and leaves media mounted and streams connected.

`power()` is the only thing that loses RAM and re-reads ROM images.

What each signal does is a fact about your board, and it comes from the manual. It is tempting to scrub the card clean on `RESET*` because it feels safe, and it is exactly backwards: **a card that resets more of itself than the reset line physically reaches is inventing a machine nobody built, and the invention is destructive.**

The 88-2SIO proves it. The **MC6850 has no RESET pin** — 24 pins, and RESET is not among them — so `RESET*` reaches that card's address decoding and *nothing else*. This tree used to reset both ACIAs anyway, throwing away the guest's word format and interrupt enables and eating a byte out of the receive register, on a card where a real bus reset would have preserved all of it.

The memory card is the model to copy: **it clears its bank latch on either reset and touches not one byte of RAM.**

Your board's `.md` must say concretely what each of the two does to it. That is a gate, not a nicety (DESIGN.md §14).

Lifecycle, time, and the host

`pump()` — the one door to the outside world

```
virtual void pump() {}
```

Give the host a turn: accept a socket connection, drain a keyboard. It is called **once per time slice by the run loop, and NEVER from inside a bus cycle.**

That seam is what keeps a board pure. `read()` and `write()` are pure computation over the card's state; anything that has to *talk to the outside world* happens here, at a known point in emulated time. That is what would let a recorded session replay identically instead of depending on when the host scheduler happened to deliver a packet.

Boards must **never** touch a socket, a file handle or `termios` directly. Everything they need from the host comes through the services in `src/host/` and `src/platform/`.

`Clock` — emulated time

Time is measured in **T-states**, never milliseconds, and it advances **only when the CPU retires an instruction, by exactly the count the CPU reported** (`StepResult::tStates`). `Machine::clock` is the one clock; a board is handed it by `attachClock()` when it goes into the backplane.

Nothing else in the simulator may call `std::chrono::now()`. That is what makes the guest's sense of time and a card's sense of time *the same sense of time* — a UART's idea of when a character has finished going out is derived from the very instruction stream the guest is timing it against, so the two cannot drift.

A card with nothing time-dependent on it never looks at the clock, and most don't. A UART absolutely does: **TDRE is a deadline, not a flag.** Two facilities, and they answer different questions:

- **`Clock::at()` / `after()`** — a *deadline*: something emulated time already knows is coming. A character finishing transmission. A sector arriving under the head.
- **`pump()`** — a *keystroke*, which is not in emulated time at all and which nothing could have scheduled.

The 6850 needs both, in the same function: `pump()` takes the byte off the line, and if the line has not yet had time to deliver it, sets a deadline for when it will.

The crystal is on the CPU card, so `clock_hz` is that board's property. There is no machine clock and there must never be one again — a machine-level copy would be a second place to say one thing, and the day the two disagreed the machine would run at whichever was written last.

`drainLog()` — what the card wants said out loud

```
virtual std::vector<std::string> drainLog() { return {}; }
```

A bank select it could not decode. A ROM that failed to load. A sector whose checksum did not match. Drained by the monitor after every command and after every run, and cleared by the draining; MCP returns the same strings as structured data.

It is on `Board`, and it is virtual, because it used to be a `dynamic_cast<MemoryBoard*>` in `Machine::drainBoardLog()` — which meant a disk controller with something to say about a bad sector **had no way to say it**. A general facility with one card's name compiled into it is a bug, every time.

A chip is not a card

`src/chips/` holds the parts that get soldered onto more than one card: `mc6850.h` (both halves of the 88-2SIO), `uart1602.h` (the COM2502 — the 88-SIO and the 88-ACR), `wd17xx.h` (the FD1771).

Each chip is modelled from its DATA SHEET. Each board is modelled from its MANUAL. A chip built instead from the one BIOS that happens to drive it implements exactly the subset that BIOS touches and quietly gets the rest wrong — and it looks finished while doing it.

A chip knows nothing about S-100. It has a clock, some pins, and (if it moves bytes) a `ByteStream`. What it talks to is an **interface, never a file**: `Wd1771` has a `FloppyDrive` — STEP, DIRC, TR00, WPRT, READY, IP, with a drive on the far end. It owns no image, no geometry and no host file, and it has **no drive-select and no side-select pin**, because the FD1771 hasn't got either. Which drive those pins reach is a **latch on the card**. That absence is the chip/board seam stated in silicon.

And the seam is not "the chip does the work and the board forwards to it." The 88-SIO's status word is **inverted** and the 88-2SIO's is not. A shared UART class with a `bool invert` on it is *precisely* the bug `src/chips/` exists to prevent — so those two cards share **no code**, on purpose. What permits sharing is being **the same part**, not filling the same role.

Where the seam falls is a fact about the chip, not a house style. The chip is what the data sheet describes, at its pins, in true sense. Everything between those pins and the S-100 bus — the inverting

buffers, which bit each signal lands on, the interrupt-enable flip-flops that live in a *different IC*, the port decode — is the **card's**, and it stays on the card.

Where this leaves you

You now know what a board is: a thing that is asked four pure questions (`assertsPhantom`, `decodes`, `assertsInt`, `assertsVi`), that is *told* two things (`read`, `write`), that may watch the backplane (`snoop`), that describes itself (`properties`), and that gets a turn to talk to the host at one known point in emulated time (`pump`). Everything else on the vtable is a detail of one of those.

You also know the traps: a decode with a side effect in it, a decode that changed without saying so, a missing `intChanged()`, a card that resets more of itself than the real one does, and a board that manufactures its own `0xFF`.

That is enough to write a board. The next chapter writes one — a lamp latch on `OUT FF`, which is free precisely because the front panel decodes `IN FF` and nothing else.

Writing a board

We are going to build a board. A real one — it plugs into the backplane, it decodes a bus cycle, it holds state, it has a setting you can put in a machine file, and when we are done the monitor will know about it, `CONFIG SAVE` will write it out, and an AI assistant driving the machine over MCP will be able to configure it. None of that last part will cost us a line of code.

The board is **eight lamps and a latch**. `OUT 0FFH` lights them. That is the whole thing.

The finished source is in the tree at **`tests/boards/lamp/lamp.h`**, and `tests/test_lamp.cpp` drives it on a real bus — so it compiles and it is tested, which is not something you can say about most tutorial code. Read along, or write it yourself.

Why port FF, which looks taken

The front panel is already at port `0xFF` — that is where the SENSE switches live. So this looks like a collision, and picking `FF` looks like a mistake.

Ask the machine:

```
altairsim> WHO IO FF
port FF OUT: nobody (an OUT here goes nowhere)
```

```
altairsim> SHOW BUS IO
I/O
  FF        fp0      read  SENSE switches SA8..SA15 -> D0..D7
  10        sio0     read/write 6850 'a' -- status/control, data
  ...
```

The front panel answers `IN FF`. It does not answer `OUT FF`. That is not a simplification in the model — it is the real card. The SENSE switch buffer's enable is gated with `sINP`; there is no `sOUT` anywhere near it. On a real Altair an `OUT 0FFH` is not the panel's, and the byte goes nowhere at all.

And the bus knows the difference, because **the bus decodes each direction separately** — `ioRead_[256]` and `ioWrite_[256]` are two different tables (`src/core/bus.h`), for the same reason the real backplane has two different strobes.

> **The bus routes by cycle type, not just by address.**

So `OUT FF` is a genuinely empty slot in a stock machine, and our board can have it without disturbing anything. Keep that sentence in mind while writing `decodes()` — it is the single easiest thing to get wrong, and getting it wrong here would break the SENSE switches.

1. The board

`tests/boards/lamp/lamp.h`. A board is a class that inherits `Board` (`src/core/board.h`) and answers some questions.

Its name

```
std::string type() const override { return "lamp"; }
```

This is the word a machine file uses: `type = "lamp"`. It is the **chip or the common name**, never a catalog number — nobody ever asked for an 88-CPU, they asked for an 8080.

What it decodes

```
bool decodes(const BusCycle& c) const override {
    if (!enabled_) return false;                // a board switched off drives nothing
    if (c.type != Cycle::IoWrite) return false;  // OUT only. The panel owns IN.
    return c.port() == port_;
}
```

That middle line is the whole lesson of this chapter. Delete it and the board answers `IN FF` too — the SENSE switches stop working, and the bus starts reporting contention with `fp0`.

Two rules about `decodes()`, and they are not negotiable:

- **It must be pure and combinational.** It answers "*if this cycle happened, would I drive the bus?*" and it does nothing else. No side effects, no counters, no latching.
- **The bus caches the answer.** It keeps one slot per port and per page, so a decode is a table lookup rather than a walk over every board in the machine. Which is why a `decodes()` with a side effect in it is a bug that will not show up for a month: it does not get called when you think it does.

If a board's decode ever *changes*, it must say so — `decodeChanged()`. (You will see below that we get that for free.)

What it does with the cycle

```
void write(const BusCycle& c) override { latch_ = c.data; }
```

We claimed the cycle, so the byte is ours.

We never override `read()`. We never say yes to a read, so we are never asked one — and an `IN FF` from our port floats to `0xFF`, which is exactly what an output-only board does.

Power and reset

```
void reset(Reset) override { latch_ = 0; }
void power() override      { latch_ = 0; }
```

These are **two different events** and a board is entitled to treat them differently. `Reset::Bus` is the RESET* line — the button on the panel. `Reset::PowerOn` is the power coming up, and it is the only thing that loses RAM. For our lamps they mean the same thing: the lights go out. For a memory board they emphatically do not.

Its state — SNAPSHOT and RESTORE

A board writes its **runtime state** so `SNAPSHOT` can save it and `RESTORE` can put it back (`DESIGN.md` §13). Chain to the base — it handles `enabled_` — then read and write your own fields in the same order:

```
void serialize(StateWriter& w) const override {
    Board::serialize(w);
    w.u8(latch_);
}
void deserialize(StateReader& r) override {
    Board::deserialize(r);
    latch_ = r.u8();
}
```

`StateWriter` / `StateReader` (`core/statefile.h`) are explicit and little-endian — `u8`, `u16`, `u32`, `u64`, `boolean`, `str`, `blob`, and `raw` for a fixed array — so a snapshot reads back byte-for-byte on every target. Three rules decide what goes in:

- **Write runtime state, not configuration.** A snapshot is RESTOREd into a machine built from the same machine file, so your straps (the port, a baud rate, a jumper) are already correct — writing them would be a second copy that could disagree. Write the registers, latches, RAM and in-flight buffers that a running machine accumulates.

- **Do not write a host handle.** A `ByteStream`, a disk image, the `Display` — those are re-opened from the (unchanged) config. But bytes that are **not on the host yet** — an uncommitted write buffer, a tape's head position — *are* your state and *do* travel.
- **Never write a `Clock::Handle`.** The event queue does not survive a snapshot. If your board sets a deadline, **re-arm it in `deserialize()`** from the state you just read — call the same `refresh()` / `arm…()` you already call when a jumper moves. Store the deadline as an absolute T-state if you need the timing back; it stays valid because the clock's time travels with it.

Its settings — and this is the part that pays

```
std::vector<Property> properties() override {
    std::vector<Property> p;
    {
        Property x;
        x.name  = "port";
        x.help  = "the port this card latches. Write-only -- an IN here is not ours";
        x.kind  = Kind::Int;
        x.radix = 16;               // ON THE WIRE -> HEX. A port is a thing the 8080 sees.
        x.min   = 0;
        x.max   = 0xFF;
        x.get   = [this] { return Value::ofInt(port_); };
        x.set   = [this](const Value& v, std::string&) {
            port_ = (uint8_t)v.i();
            return true;
        };
        p.push_back(std::move(x));
    }
    {
        Property x;
        x.name  = "lamps";
        x.help  = "what the guest last wrote -- the eight LEDs";
        x.kind  = Kind::Int;
        x.radix = 16;
        x.get   = [this] { return Value::ofInt(latch_); };
        // NO SETTER.
        p.push_back(std::move(x));
    }
    return p;
}
```

This vector is the entire configuration layer. There is no schema file, no parser, no registration call, and nowhere else to declare anything. `SET`, `SHOW`, the TOML loader, `CONFIG SAVE`, the MCP tool schemas, tab completion, and the User Manual's generated board reference are all written **once**, against this, and know nothing about any particular board.

Two details worth stealing:

- **`radix = 16`**, because a port is a thing the processor can see. On the wire → hex; never on the wire → decimal. Get this wrong and `port = 10` in a machine file quietly means ten instead of sixteen. **It declares the class, not a display preference** — the one `radix` both reads bare digits and prints them, an operator can force any base of their own (`0x`, `0o`, `%`, `#`) whatever you chose, and `SET CONSOLE base=octal` does not reach it: that switches how the *monitor* prints addresses and bytes, while `SHOW` prints every property in its own radix. So pick it by which side of the wire the number lives on, and never because one base reads better.
- **`lamps` has no setter**, and that absence *is* the signal. `SHOW` prints `(read-only)`, `CONFIG SAVE` leaves it out of the file, and the manual's reference marks it. A setter that merely *refused* would stop a `SET` and fool all three at once — which is a mistake that was in this codebase, on the memory board, until this manual went looking.
- **We never call `decodeChanged()`** in the `port` setter. The property layer calls it for us after any successful set, precisely so that a board author cannot forget.

What it tells the operator

```
std::vector<MapEntry> ioMap() const override {
    return {{port_, port_, "write", "lamp latch -- D0..D7"}};
}
```

This is what `BOARDS`, `SHOW BUS IO` and `WHO` print. **It is documentation, not decode** — the bus never consults it. A board whose `ioMap()` disagreed with its `decodes()` would work perfectly and lie to you, which is worse than a board that does not work.

`memMap()` is the same thing for memory, with one more rule: **it says how the board is built, and it does not follow a latch.** `SHOW MACHINE` prints it, and how a machine is built does not change when the guest writes a port. A board with a PROM that the guest can switch out lists the PROM always. `SHOW BUS MAP` asks the bus which of those addresses the board answers now, and takes only the text from `memMap()`, so the board needs no code for that.

A board with a PROM, or with any memory that a read does not change, also gives `peek()`. Without it, `DISASM` and `DUMP` show `FF` there while the processor reads the correct bytes.

2. Put it in the registry

Two lines and an include, in `src/boards/registry.cpp`:

```
#include "../../tests/boards/lamp/lamp.h"

// ...in boardTypes():
{"lamp", "Eight-LED output latch (the Developer Guide's example)",
         "A write-only latch: OUT FF lights eight LEDs. The Developer Guide builds this"},

// ...in makeBoard():
if (type == "lamp") return std::make_unique<LampBoard>();
```

The row is the type name, a one-line summary for the `SHOW BOARDS` catalog (keep it short enough to fit the column -- a test checks every row fits 78 columns), and the full description that `SHOW BOARD <type>` prints.

That is all. `registry.h` promises "*adding a board type is one line here and nothing anywhere else*", and it means it.

(Our board is a header, so it needs no entry in `CMakeLists.txt`. A real one — with a `.cpp` — adds its source to the `altair_core` library alongside the others.)

3. Build it, and watch what you get for free

```
$ cmake --build build -j
```

The board is now in the catalogue. `SHOW BOARDS` lists every type with its one-line summary; name one to see its settings, their help text and their legal values -- and nobody wrote a line of code to put it there:

```
altairsim> SHOW BOARDS
  TYPE         DESCRIPTION
  -----------  ------------------------------------------------------------
  ...
  lamp         Eight-LED output latch (the Developer Guide's example)
  ...

  SHOW BOARD <type> for a board's properties (add UNITS for just the units)

altairsim> SHOW BOARD lamp
  lamp  A write-only latch: OUT FF lights eight LEDs. The Developer Guide
        builds this

  PROPERTY  HELP
  --------  ------------------------------------------------------------
  port      the port this card latches. Write-only -- an IN here is not ours
            values: 0x0 .. 0xFF
            default: 0xFF
  lamps     what the guest last wrote -- the eight LEDs
```

Fit one, and ask the machine the same question we asked at the start:

```
altairsim> BOARDS ADD lamp lamp0
lamp0: lamp added

altairsim> WHO IO FF
port FF OUT: lamp0
```

It said `nobody` an hour ago. Now light the lamps:

```
altairsim> OUT FF 55
port FF <- 55

altairsim> SHOW lamp0
lamp0  (lamp)

  property         value            legal
  port             0xFF             0x0..0xFF
  lamps            0x55             (read-only)
```

`OUT FF 55` ran a **real output cycle on a real bus** — the same path the 8080's `OUT` instruction takes, because there is only one. The board decoded it, latched it, and `SHOW` read it back out of the reflection layer. `lamps` is marked read-only, and it worked that out from the missing setter.

And the front panel is still sitting on the same port, untroubled:

```
altairsim> IN FF
port FF -> 00      <- still the SENSE switches
```

One port. Two boards. No contention. **Because the bus routes by cycle type.**

4. Put it in a machine

```
[machine]
name = "lamps"
base = "default"

[[board]]
type = "lamp"
id   = "lamp0"
port = FF          # radix 16 -- no 0x needed
```

Nobody taught the TOML loader what a lamp is. It asked the board for its properties, found one called `port`, and set it. A board added next year is configurable, scriptable, drivable by an assistant, and documented **the day it lands**.

5. Test it

`tests/test_lamp.cpp` is the model. It does three things, and the middle one is the important one:

```
// A REAL OUT CYCLE on the bus -- not a method call on the board.
m.bus.ioWrite(0xFF, 0x55);
CHECK(lamp->properties()[1].get().i() == 0x55, "OUT FF latches the byte");

// The board is WRITE-ONLY, and the proof is that the read FLOATS.
CHECK(m.bus.ioRead(0xFF) == 0xFF, "an IN from FF is not the lamp's -- the bus floats");
```

…and then it pins down the claim this whole chapter rests on, with both boards in one machine:

```
BusCycle in {Cycle::IoRead,  0xFF};
BusCycle out{Cycle::IoWrite, 0xFF};

CHECK(m.bus.respondersTo(in).size()  == 1, "IN FF: the front panel, and ONLY the front panel");
CHECK(m.bus.respondersTo(out).size() == 1, "OUT FF: the lamp, and ONLY the lamp");
CHECK(m.bus.drain().empty(),               "...and the bus says NOTHING. It is not contention.");
```

Assert the thing your design depends on, not the thing that is easy to assert. If the bus ever stopped decoding by direction, this chapter would be teaching a falsehood — and that test is what would say so.

> **One trap, and it caught this test.** `m.bus.attach(board)` wires a board to the backplane; it does not hand it to the **machine**. Lifecycle events — `reset()`, `power()`, `pump()` — are dispatched by `Machine` over the boards it *owns*. A board that is only on the bus will answer cycles all day and never hear the RESET line. A board that comes from a machine file goes in through `Machine::add()` and gets both.

6. What the lamp skipped — wiring a board that ships

The lamp is complete, tested, and configurable, and everything above is the whole story for a board that only latches a byte. A board that talks to the **outside world**, or that a user will find in the manual, needs a few connections the lamp never made. None of them is hard; each is easy to *forget*, and the forgetting fails in a way that does not point back here.

A board that talks to an endpoint wires its resolver in two places

A board with a real line — a serial port, a printer, a socket — does not open that line itself. It hands a *spec* like `socket:2323` or `file:printout.txt` to a resolver and gets back a `ByteStream`. **The endpoint grammar lives in exactly one file, `src/host/endpoint.cpp` (`resolveEndpoint`), and a board must not know it** — that separation is what lets `CONNECT`, tab completion and the MCP schema all speak the same vocabulary without any board learning what a socket is.

A board reaches the resolver through a static `setResolver()` (the 88-C700 and the SIO family are the models). The trap is that this is wired at the **composition root**, and there are **two** of them:

- `src/main.cpp` — the shipping program.
- `tests/main.cpp` — the test harness.

> **Wire the resolver in `tests/main.cpp` too, or your board test connects to nothing.** Both files carry the same block of `YourBoard::setResolver(resolveEndpoint)` lines. A board added to only `main.cpp` works in the running program and then every test that `CONNECT`s it gets a null resolver and fails somewhere unhelpful. Copy the line into both.

Remember the path as written, not as resolved

If your endpoint is a path (`file:`, a disk image), resolve it through the board's `resolvePath()` so a relative path means the right thing (relative to the machine file when it came from one, relative to the shell when typed — the rule is in the serial-I/O chapter and the config docs). But **store the spec as the user wrote it** for `describe()` and round-trip.

> **Where you LOOK is `resolvePath()`; what you REMEMBER is the path as written.** Save the resolved absolute path instead and `CONFIG SAVE` saves *that* back, so the next load rebases an already-absolute path and the one after that rebases again. The hard-sector controller states the rule in a comment at its `openMedia` site; follow it.

Adding a *new* endpoint scheme touches its help in two spots

If you are not just consuming endpoints but adding one (a new `something:` scheme in `endpoint.cpp`), it must appear in **`endpointHelp()`** *and* get a one-line explanation in the `CONNECT` command help in `src/cli/commands.cpp`. `test_cli.cpp` asserts that every name `endpointHelp()` offers is a word `CONNECT` explains — a bare `null` or `scripted` tells a user nothing, and the explanation is a hand-copy that rotted once (it promised `socket:` "was coming" long after it shipped). The test is what keeps the two honest.

A board with a speaker holds a `Speaker`

A board whose sound is a latch into a D/A — the guest writes a level in a timed loop — does not render PCM itself. It holds one `Speaker` (`src/host/speaker.h`) for each voice and calls it in two places:

```
void write(const BusCycle& c) override {          // the bus cycle only records
    if (c.data != latch_ && clock_) spk_.edge(clock_->now(), (int8_t)c.data);
    latch_ = c.data;
}
void pump() override {                            // the host turn makes the sound
    if (clock_) spk_.pump(*clock_, true);
}
```

Also call `spk_.clear()` in `power()`, `spk_.resync(now, level)` in `deserialize()`, and put `spk_.status(clock_)` in `statusLines()`. The host `Audio` service is already wired for every `Speaker` (`Speaker::setAudio` in both composition roots), so there is nothing to add there. The Cromemco D+7A and the Newtech Model 6 are the two examples; DESIGN.md §7.4 has the rules, and the first is that **a machine with no crystal plays nothing**.

A board that holds READY calls `holdReady()`

If the real board pulls READY low and makes the CPU wait, call `holdReady(n)` from `read()` or `write()`, with the hold in T-states for that cycle. The run loop adds it to the instruction's time. If the manual gives the hold in microseconds, convert it at `clock_->hz()`, so a faster CPU waits more states. Do not keep the wait count yourself, and do not advance the `Clock` yourself. The D+7A is the example (`analogHold()`), and DESIGN.md §7.5 has the rule.

Regenerate the reference, ship a machine, and mind the two unguarded docs

Three loose ends after the board itself compiles:

- **The generated board reference.** After editing `registry.cpp`, `commands.cpp`, or a machine file, regenerate `docs/manual/ref/*.md` — `cmake --build build --target docs-reference` — and commit the result. A ctest byte-diffs the committed files and goes red until you do. **Edit the emitter or the source, never the `.md`.**
- **A sample machine is a TOML file**, not C++: drop it in `machines/` with `base = "default"` under `[machine]` and a `name`. `cmake/embed_machines.cmake` embeds it on the next build.
- **Two documents no test guards, so they drift silently.** The prose board chapter (`docs/manual/boards.md`) and the changelog's `Unreleased` section are *not* test-enforced — the manual guard only checks self-containment and `ORDER` membership, nothing checks that the prose lists every registered board. The authoritative list is `registry.cpp`'s `boardTypes()`; the generated `ref/boards.md` is guarded, but the hand-written chapter is not, and it once drifted nine boards behind before anyone noticed.

> **When you add a board, update the prose chapter and add an `Unreleased` changelog line by hand. No build will remind you.** A release removes the `## Unreleased` header, so add it above the newest release if it is not there. The board also ships with its own `docs/boards/*.md` (from `docs/boards/_TEMPLATE.md`) and a row in `docs/sources.md` for anything it embeds.

What to do next

Our board answers a cycle. A more interesting one **asks for something**:

- **Interrupts.** Add an `IrqJumper` and push `irqJumperProperty("interrupt", ..., irq_)` into `properties()` — that one call buys you the whole `none | int | vi0..vi7` vocabulary, tab completion, and the flag that lets `SHOW BUS IRQ` find your strap. Then override `assertsInt()` /

`assertsVi()`, and **call `intChanged()` from every place your pending flag could move.** A needless call costs a virtual call. A missing one hangs the guest forever.
- **Time.** A board with a deadline uses the `Clock` it was handed. A UART absolutely needs one: transmit-buffer-empty is a deadline, not a flag.
- **The outside world.** Anything that talks to a socket, a file or a keyboard does it in `pump()` — **never inside a bus cycle**. That seam is what keeps `read()` and `write()` pure computation over state, and it is what a deterministic replay would be built on. The **88-C700 printer** (`src/boards/mits-88c700.{h,cpp}`, `docs/boards/mits-88c700.md`) is the smallest shipped board that does this for real: a bare latch that sinks its data port to a `ByteStream`, so `CONNECT lpt0:prn file:printout.txt` captures the printout and the board never learns what a file is. It is a good next read after this lamp.
- **Sub-units.** A board with a *list* of things — regions on a memory board, drives on a controller — declares `subUnitTables()` and gets `[[board.region]]` / `[[board.drive]]` in TOML for free.

And whatever you build: **it ships with its `.md`.** A board and its documentation are one deliverable, and the **Limitations** and **Quirks** sections are the load-bearing ones — they are what forces the honest question "*what did I not actually implement?*", which is exactly the question a simulator author most wants to avoid. `docs/boards/_TEMPLATE.md` is the form.

Writing a video board

The lamp chapter built a board that latches a byte. This one builds a board that **paints** — and it is the same board with one more connection: a host video service it draws into. The worked example is **CADzilla** (`src/boards/cadzilla.{h,cpp}`, `docs/boards/cadzilla.md`), an HD63484 ACRTC with a Bt453 RAMDAC, chosen because it exercises every part of the seam: a chip with its own frame memory, a chip that *is* the palette, and a picture you can only reach by issuing commands. The Dazzler and the VDM-1 are the two simpler shapes and are cited where they differ.

Everything here is tested with **no window**. That is not a limitation to work around; it is the design, and the last section is how you prove a picture without one.

1. The nine questions

A video board is an ordinary `Board` that happens to paint, and nine decisions turn into code. Answer them in the board's `docs/boards/*.md` **before** writing the class; seven become a method each, and the

other two are the Limitations section.

#	Question	CADzilla's answer	Where it lands
1	**Name.** The chip or the common name, never a catalog number. "board", never "card" (`DESIGN.md` §0.3)	`cadzilla`	`type()`, the registry, the filename
2	**I/O footprint** — which ports, and *separately* which answer `IN` vs `OUT`: the bus decodes by cycle type	one 8-port block from `port`: `+0` / `+2` the ACRTC (both directions, different registers each way), `+1` the board's own MODE register (write-only), `+4` .. `+7` the Bt453 (both directions); `+3`, the high byte of a 16-bit transfer an 8-bit host never makes, answers nothing	`decodes()`, `ioMap()`
3	**Memory footprint.** On-board screen RAM the CPU addresses (VDM-1 decodes a 1 KB window), a framebuffer in main RAM (Dazzler decodes none, reads RAM), or private memory (CADzilla decodes none, and the CPU never sees it)	none	`decodes()`, `memMap()`
4	**Where pixels come from and how you know they moved.** A write to on-board RAM latches a dirty flag; main RAM has to be polled against a shadow; a chip with its own memory tells you	`Hd63484::takeDirty()`, `Bt453::takeDirty()`	`pump()`
5	**Geometry.** Native w×h — fixed, decoded from registers each frame, or **the monitor's**: a board with a real CRT controller can carry a fixed-frequency display and place the chip's picture in it. `PixelFormat::Indexed8` is the only format the seam has	the `mode` strap's VESA frame (one of the three primary VESA resolutions, 1024x768 by default); the ACRTC's picture placed by HDS/VDS against the mode's porches	`render()`, `acquire()`
6	**Palette.** How many entries, from where. Dazzler: 16 from an RGBI nibble. VDM-1: 2. CADzilla: **256, and they are a chip**	`Bt453::palette()`	`setPalette()`
7	**Observable timing.** A status bit a guest can *time* (vblank, scan parity) comes off `clock_->now()` — never a poll counter. An oscillator the guest cannot observe (a cursor blink) comes off `Display::hostSeconds()`	the drawing time at `draw_rate=real`: T-states carried into 2CLK (`sync()`, remainder kept), one Clock deadline at the command's end (`arm()`) so an interrupt lands with no bus cycle; the raster counter is not modeled	`sync()`, `arm()`

8	**Straps vs live status.** A strap has a setter and round-trips through `CONFIG SAVE`; live status has **no setter**, and that absence is the whole signal. A register that is write-only *on the wire* (a chip's format byte, or a board's own glue register) is still reflected as read-only live status — the board keeps the shadow the wire cannot give back. A fixed hardware fact with no jumper on the board (CADzilla's 2 MB of SRAM) is neither a strap nor live status — it is a constant. Every video board pushes `Display::widthProperty(videoWidth_)`	`port`, `mode`, `draw_rate`, `width`, `interrupt`; live `video`, `picture`, `wiring`, `hspol`/`vspol`/`amode`/`olen`, `status`, `irq`	`properties()`
9	**Snapshot.** Runtime state only — never a strap, never the `Display*`. If no memory board holds your pixels, **they travel with you**	both chips, frame memory as a `blob`	`serialize()`

Two things that are *not* the board's business, and each is the classic mistake: a **keyboard** is a separate board or an endpoint, never the window (`host/display.h`'s header note); and the **frame rate** is the host's (`wantsFrame()`), never yours.

2. Chips first, if the board has any

CADzilla's two parts are chips before they are a board, in `src/chips/`, and that split is the one the FD1771 and the 8257 already made (`chips/wd17xx.h`, `chips/i8257.h`): a chip knows nothing about S-100 or about the board's ports. It has *pins* — the ACRTC's RS, the Bt453's C1C0 — and the board decodes an address onto them:

```
uint8_t CadzillaBoard::read(const BusCycle& c) {
    uint8_t off = (uint8_t)(c.port() - port_);
    if (off == 0 || off == 2) return acrtc_.read(off == 2);   // RS: +0 status, +2 data/FIFO
    return dac_.read(off & 3);                                // C1C0 = A1A0
}
```

The pay-off is that a chip is tested on its own, from its data sheet, with no bus at all — `tests/test_hd63484.cpp` and `tests/test_bt453.cpp` — and the board test is left with only the board's own decisions to prove. The **decisions a chip cannot make** are the board's, and they belong in the board header where CADzilla's are: the shift register (8 bits per pixel, 8 words per fetch, low byte first), the monitor (a fixed VESA frame), how much frame memory was fitted, what the overlay inputs are tied to, where IRQ* is strapped. Draw the line where the hardware draws it: the ACRTC only ever puts an *address* on its bus, so `Hd63484` answers address-level questions (`backgroundRaster()`, `windowRaster()`, `gaiWords()`) and the *board* fetches words and makes pixels of them — which is also why a program that sets the chip to 4 bpp gets a scrambled picture here, as it would on the board, instead of a helpfully re-unpacked one.

3. The board: three gates and one call

The whole host side of a video board is `pump()`, and it is three gates in a fixed order:

```
void CadzillaBoard::pump() {
    if (!g_display) return;                     // no service at all (the bench)
    bool a = acrtc_.takeDirty();                // consume BOTH flags every time, so one
    bool d = dac_.takeDirty();                  //   is never lost behind the other
    dirty_ = dirty_ || a || d;
    if (!dirty_) return;                        // deterministic: would it look different?
    if (!g_display->wantsFrame()) return;       // host-side rate limit; 0 = always, in tests
    render();
    dirty_ = false;
}
```

The order matters. Change detection is **deterministic** and always on: it is what makes a headless test reproducible. The frame limit is **wall-clock** and off in tests: it is a host economy that changes what is *drawn*, never what is *computed*. Put them the other way round and a test's frame count depends on the machine it ran on.

`render()` is one call that is the window, and its arguments are not decoration:

```
Surface* s = g_display->acquire(this, id, m.width, m.height, PixelFormat::Indexed8, videoWidth_);
```

`this` keys **this board's own window** (issue #234: two boards of the same resolution would otherwise land on one Surface), `id` **titles** it, `videoWidth_` **sizes** it — and `m` is the monitor mode, so the frame is the monitor's, not the chip's. Get that line right and the SDL back end's windowing, integer scaling, CRT look, focus policy and close box all arrive with no further code — and a `NullDisplay` keeps the same Surface per owner for a test to read. Then paint every pixel (the Surface may be last frame's buffer), hand over the palette, and present:

```
g_display->setPalette(this, dac_.palette());    // the RAMDAC IS the palette, verbatim
s->clear(0);                                    // blanking is black
if (on) paintFrame(s, m.width, m.height);       // the shift register, over the ACRTC's addresses
g_display->present(this, s);
```

That is the seam's whole shape, and for a board with a RAMDAC it is not an abstraction: an `Indexed8` Surface *is* the eight-bit pixel bus and `setPalette()` *is* the look-up table. Nothing gets translated.

4. Wiring — every file a video board touches

File	What goes in
`CMakeLists.txt`	the `.cpp`s in `altair_core`; the test in `altair_tests`
`src/boards/registry.cpp`	`boardTypes()` and `makeBoard()`
`src/main.cpp` **and** `tests/main.cpp`	`YourBoard::setDisplay(&g_display)` — miss the second and the board draws into nothing in every test
`tests/test.h`, `tests/main.cpp`	the suite's declaration and its table row
`machines/<name>.toml`	a sample machine
`docs/boards/<name>.md`	from `_TEMPLATE.md`; Limitations and Quirks are the load-bearing sections
`docs/manual/boards.md`, `docs/changelog/changelog.md`	the prose list and an `Unreleased` line (add the `## Unreleased` header if it is not there) — **no test guards either**
`docs/manual/ref/`	regenerated: `cmake --build build --target docs-reference`

Nothing in the bus, the monitor, the TOML loader or the MCP server changes. `properties()` is the entire configuration layer.

5. Proving a picture with no window

`test_dazzler.cpp` used to check a picture one pixel at a time — `px(32, 0) == 1` — which is fine for "is this element red" and useless for "is this the picture", and prints nothing you can look at when it fails. Two pieces fix that.

`host/framedump.h` takes a Surface and its palette out of the seam in the forms a person and a test can use: `frameText()` renders it as a **text grid**, one character per pixel from a legend; `framePpm()` / `writePpm()` resolve it through the palette into a **PPM image** any viewer opens; `frameCrc()` hashes the resolved RGB, so a palette-only change moves it. For a board with a RAMDAC, `frameRgb()` is literally the DAC in software — what the wire carries.

`tests/framecheck.h` builds the assertion on top. The expected picture is a raw string literal *in the test*, readable by a person; on a mismatch the failure prints the first row that differs with a caret under the column **and writes what the board drew as a `.ppm`**, naming the path. A 640x480 frame is not readable one character per pixel, so CADzilla's end-to-end test draws its rectangle, line and dot on a 32-pixel grid and samples the frame at the same pitch — 20 x 15 characters that a person can check against the commands that drew them:

```
TextGridOpts every32;
every32.xStep = 32;
every32.yStep = 32;
CHECK_FRAME_OPTS(g.disp, g.cad, R"(
1111111111111111111
1.................1
1.................1
1.................1
1.................1
1.................1
1.................1
1.3333333333333333.1
1.................1
1.................1
1.................1
1.........2........1
1.................1
1.................1
1111111111111111111
)", every32, "sampled every 32nd pixel: the rectangle, the line and the dot, right way up");
```

Every character is a pixel's palette index. A rectangle one row too high, a mirrored X axis, a picture placed one memory cycle late — each is visible in the diff, and the `.ppm` beside it shows the full-resolution picture through the colors the guest loaded.

Be clear about what a sampled grid proves. It looks at one pixel in a thousand: it proves the *geometry* in a form a person can read, and nothing about the pixels between the samples. So pair it with **`CHECK_FRAME_PIXELS`**, which compares **every** pixel against an oracle the test writes — a function from `(x, y)` to the palette index the commands must have left there. For a rectangle, a line and a dot that is four comparisons:

```
auto oracle = [](int x, int y) -> uint8_t {
    if (x == 320 && y == 352) return 2;                                  // the dot
    if (y == 224 && x >= 64 && x <= 544) return 3;                       // the line, end excluded
    bool onRect = (y == 0 || y == 448) ? (x <= 608)                      // top and bottom edges
                                       : ((x == 0 || x == 608) && y < 448);   // the sides
    return onRect ? 1 : 0;
};
CHECK_FRAME_PIXELS(g.disp, g.cad, oracle, "all 307,200 pixels match the oracle");
```

A single wrong pixel anywhere fails it, and the failure names the first differing pixel, how many differ in all, and the `.ppm`. The grid is for the reader; the oracle is for the proof. Use both. For a frame you cannot describe in a function, keep a golden file under `tests/golden/` and `CHECK_FRAME_GOLDEN` it; the golden is rewritten only under `ALTAIR_TEST_WRITE_GOLDEN=1`, because a golden that rewrites itself asserts nothing.

Then prove the two things a picture alone does not: that a **palette-only change** reaches the host (`frames()` advanced, `frameCrc()` moved, the pixel bytes unchanged), and that a **snapshot** repaints the same grid on a fresh board. Both are in `tests/test_cadzilla.cpp`.

What CADzilla left for next time

Its own `docs/boards/cadzilla.md` says exactly what is not modeled. **Interrupts** are wired — `IrqJumper` and `irqJumperProperty()`, `assertsInt()` / `assertsVi()` reading the chip's own `irq()`, and `intChanged()` called from every place that could move, per the lamp chapter's closing section — so that recipe step is already done here and is the worked example for the next board that needs it. **Drawing time** is done too, at `draw_rate=real`: the chip counts only 2CLK and is told the time (`Hd63484::advance()`), and the board owns the conversion from T-states and the one Clock deadline — the pattern for a chip whose speed depends on a clock the board supplies. One piece still remains: **the raster counter** (`r80` reads 0). A guest reads it to sync to the beam, so it must come off the `Clock`, like the Dazzler's vblank bit; the frame's slot timeline that `Hd63484::buildTimeline()` already builds for drawing time is where its position would come from.

Serial I/O

Every card that moves characters — the 88-2SIO, the 88-SIO, the ACR cassette, the line printer, the paper-tape reader, the PMMI modem, the Turnkey's onboard 6850 — is built on the same three layers, and the whole point of them is that a card only ever touches the top one.

> **A board knows it has a serial line. It does not know, and must never learn, what is on the other end of it.**

That one sentence is why `CONNECT sio:a socket:2323` needs no code in the 2SIO, why a card written next year is cross-platform, scriptable and replayable for free, and why there is a whole chapter here instead of a paragraph in each board's source.

The three layers, top to bottom:

Layer	Who	Owns
The **card**	a `Board` (`src/core/board.h`)	port decode, register bits, polarity, interrupt straps — everything the *manual* describes
The **stream**	a `ByteStream` (`src/host/stream.h`)	non-blocking byte movement, the modem pins, the line rate
The **endpoint**	`resolveEndpoint` (`src/host/endpoint.h`)	the grammar: `console`, `socket:2323`, `serial:`, `in:` / `out:`, …

A card reaches down to the stream. It **never** reaches down to the endpoint — it does not know the grammar, and it must not learn it.

The ByteStream contract, and the one discipline

`src/host/stream.h` is the seam. Read it; it is short and every comment in it is load-bearing. The interface a card actually uses:

```
virtual size_t read(uint8_t* buf, size_t n) = 0;   // NON-BLOCKING
virtual size_t write(const uint8_t* buf, size_t n) = 0;
virtual bool readable() const = 0;  // -> drives RDRF / RxRDY
virtual bool writable() const = 0;  // -> drives TDRE / TxRDY
virtual std::string describe() const = 0;   // round-trips the operator's spec
```

The discipline is one line and the whole subsystem stands on it:

> **THE BOARD NEVER BLOCKS.** It asks `readable()` / `writable()` and moves on.

A UART that blocked would stop emulated time, and stopping emulated time to wait for a human is precisely how an emulator comes to drop characters. So `read()` returning `0` is **not an error and not an end of file** — it is a quiet line, a poll that found nothing, exactly what an `IN` from a real UART's data port gives you when nobody has typed. A stream never signals EOF up into a board; a board polls forever, because that is what the silicon does.

Two more methods complete the contract, and both exist so a board stays pure:

- **`pump()`** is called once per time slice **by the run loop, never by the board**. It is the one place a stream may talk to the host: accept a socket, drain a keyboard, poll a file. A card's `pump()` is a one-liner that forwards to `stream_->pump()`. Everything a board does in `read()` / `write()` is pure computation over its own state; anything that touches the outside world happens here, at a known point in emulated time. That seam is what would let a recorded session replay identically.
- **`drainLog()`** is what the far end wants said out loud — a print job that could not be spooled, a serial port that cannot do the baud it was asked for. The board drains it off its units' streams (the default `Board::drainLog()` does exactly this) and the monitor prints it after every command and run.

The unconnected line is not a special case

```
class NullStream : public ByteStream {
    bool readable() const override { return false; }
    bool writable() const override { return true; }   // TDRE set forever
    size_t write(const uint8_t*, size_t n) override { return n; }  // consumed, gone
};
```

An unconnected unit is bound to a `NullStream` — TDRE set, RDRF clear, forever, and a write goes nowhere without complaint. That is exactly how an unconnected 6850 sits on a real card. Because of it there is **no null pointer anywhere in a board's stream path, and therefore no `if (nothing is plugged in)` branch in any board.** Bind the stream to a `NullStream` in the constructor and never let it be null again.

Two more streams are worth knowing because they are how you test:

- **`LoopbackStream`** jumps TX to RX *and loops the modem pins back too* (RTS→CTS, DTR→DCD/DSR). It is the one endpoint that tests modem control with no hardware — the plug in the drawer.
- **`ScriptedStream`** keeps the two directions **separate**: a test types into `feed()` and reads what the guest printed out of `out()`. This is what makes a guest program a unit test — a monitor's banner and its answer to a command are just bytes it wrote to a serial port, and a test that can see them

asserts on them with no terminal, no thread, and no timing in the picture. It also counts empty polls (`hungry()`), which is how the MCP run loop knows a prompt is spinning on input versus a loader that is merely quiet while it works.

The endpoint grammar lives in exactly one place

```
std::unique_ptr<ByteStream> resolveEndpoint(const std::string& spec, std::string& err);
```

`src/host/endpoint.cpp` is the **only** file in the program that knows the endpoint grammar — `console`, `socket:PORT`, `socket:HOST:PORT`, `serial:DEVICE`, `pty`, `null`, `loopback`, `in:PATH`, `out:PATH`, `file:PATH`, `printer:QUEUE`. `CONNECT` and `MOUNT` are generic monitor commands, not per-board ones (DESIGN.md §7.7): the monitor opens the endpoint, the board decides what the bytes mean. That division is why a serial card gets every backend the day it lands without one line changing in the monitor.

It never guesses. `CONNECT sio:a consle` is an error carrying the list of what it could have meant, not a silent `NullStream` that leaves you wondering why the terminal is dead.

Two helpers exist so a board can *use* the grammar without *reimplementing* it:

- **`rebaseEndpointPaths`** / **`rebasingResolver`** rewrite only the PATH-bearing specs (`in:` / `out:`) so a machine-file relative path is resolved against the machine file's directory, not the shell's cwd. The grammar of *which* specs carry a path stays in `endpoint.cpp`; the board only supplies its config dir. Remember the path **as the operator wrote it** for `describe()`, and rebase only the copy you hand the resolver — otherwise a relative path double-rebases on `CONFIG SAVE` + reload.
- **`parsePort`** / **`parseHostPort`** let a board validate a `HOST:PORT` string (the PMMI's `dial` / `answer` settings do this) using the same split the resolver uses.

The capture tap is a decorator ByteStream

`ENDPOINT|FILE` taps a line to a hex log — a poor man's protocol analyzer. It is worth studying because it shows how far a `ByteStream` decorator gets you with **zero** board or monitor changes.

`TeeStream` (`src/host/tee_stream.h`) wraps an inner stream, copies every byte past in both directions to a log file, and forwards everything else — `readable` / `writable`, the modem pins, the line rate, `pump`, `pacesItself` — unchanged. It is the same shape as `FilterStream` (`src/host/filter.h`), with one difference that matters:

> A **filter mutates bytes**, so there is exactly one of them and it lives on the console (a `strip7out` on a binary transfer corrupts it silently). A **tee never touches a byte** — it observes and forwards — so it is 8-bit clean by construction, and the "one filter, on the console" rule does not apply. You may tap *any* line: a socket, a real serial port, a tape.

The grammar is a single character. `resolveEndpoint` checks for a `|` *before* the prefix dispatch (so `socket:23|cap.hex` is not mistaken for a socket spec), splits on the first one, recurses on the left for the wrapped stream, and parses the right as `FILE[?opts]`. Because the inner is resolved by the same function, the tap composes with everything — `in:tape.tap?cps=300|trace.log` taps a paced paper-tape reader. `describe()` returns `inner->describe() + "|" + fileSpec`, so the whole tap round-trips through `SHOW` and `CONFIG SAVE`, and the `|` re-triggers the branch on reload. Two consequences fall out of the grammar living in one place: `rebaseEndpointPaths` had to learn to split on `|` and rebase **both** sides (a machine-file relative log path is config-relative like any other), and the timestamps come from an **injectable host wall clock** (the printer/tape pattern), never the emulated `Clock` — a trace whose timestamps freeze at a monitor prompt is a trace of nothing.

`fmt=text` is a transcript, not a trace. It writes the bytes the inner line *accepted* on `write()`, raw, and flushes at once so `tail -f` is live. It writes no header, no timestamps, no pin edges and no read side — the guest echoes what it reads, so logging RX would double each typed key (the mirror's rule, for the same reason). The resolver opens the file in binary mode for this format only, so Windows does not turn LF into CR LF, and refuses `ts` / `width` / `gap` / `pins` with it instead of ignoring them. `append` (any format) opens the file to add to it.

Sinks stack, and the spec splits on the LAST `|`. `scripted|socket:2323|s.log?fmt=text` is a tap on a mirror on a scripted line: the last sink is the outermost decorator and the left side recurses. `resolveEndpoint` and `rebaseEndpointPaths` both use `rfind('|')`. This is what `--mcp --mirror … --log …` builds (`mcp/server.cpp`, `console()`): Filter → Tee → Mirror → Scripted. The MCP server adds `append` because `console()` runs again after a CONFIG LOAD; `main.cpp` truncates the file once at startup. Anything that peels decorators to reach the scripted line (`asScripted`, `mcpScriptedConsole` in the monitor) must peel `TeeStream` too.

A mirror's sink is a `MirrorSink`. `MirrorStream` (`host/mirror_stream.h`) owns the rules — output is dropped with no watcher, the queue is bounded and drops its oldest bytes, typed keys are injected, `ro` discards them — and the sink is only the wire: `poll()`, `attached()`, `read()`, `write()`. `SocketMirrorSink` is the TCP listener; `PtyMirrorSink` wraps `platform::Pty` (`|pty`, `|pty:LINK`, issue #683). `SerialMirrorSink` wraps a `platform::SerialPort` (`|serial:DEVICE[?baud=N]`). A new sink is one more class and one branch in the resolver.

The serial sink is not a `HostSerialStream`. `HostSerialStream` is a line for a board: it owns the modem pins, a board reprograms it, and it accepts every byte into its own queue because a board must never see a

short write. A mirror has no board, so the sink opens the port at 8N1 and 9600 baud (or `?baud=`), with flow control off, and `write()` returns what the driver took. `MirrorStream` keeps the rest in its own bounded queue. `attached()` is always true: DCD and DSR are not reliably driven by a terminal, so nothing says when one is there. Reads and writes that fail return 0, as in `HostSerialStream`, so an adapter that is unplugged makes the mirror go quiet without an error. The test uses a fake `SerialPort`, and one case uses the slave of a pty as the device.

`pty` alone is the line itself, not a mirror. `PtyStream` (`host/pty_stream.h`, issue #685) holds the same `platform::Pty` as a plain `ByteStream`, and is built like `TcpStream`: a program that has the link open is carrier (DCD and DSR), and a full kernel buffer drops CTS, so the guest waits and loses no byte. One thing differs from the socket. With nobody on the line, CTS stays up and `write()` discards the bytes, so a board with wired CTS does not stop on a line that nobody reads. The name of the link reaches the operator two ways: `CONNECT` prints `takeNote()` at once, and a machine file gets it through `drainLog()`.

`Machine::drainBoardLog()` drains each serial line too. A board that overrides `drainLog()`, as the 2SIO does, returns the messages of its chips and never asks the stream. The machine therefore walks `units()` and `unitStream()` after each board, and prints what the stream has to say as `id:unit: message`. A board that uses the default `Board::drainLog()` has already taken those messages, and the walk finds none.

The pseudo-terminal has three traps, all in `platform/posix/pty_posix.cpp`. (1) A master whose slave was never opened reads `EAGAIN`, the same as "open and quiet", so `openPty()` opens and closes the slave once; after that `POLLHUP` means nobody is there. (2) macOS puts the slave back in cooked mode with echo at every reopen, and an echo would return the guest's output as typed keys, so `poll()` reads the settings through the master on every call while a program is attached, and sets raw mode when it finds echo or line mode. It does not wait for the edge where a program arrives: a close and a reopen between two polls show no edge (issue #687). (3) The name is a symbolic link claimed with `symlink()`, which fails if the name exists, so two simulators cannot take the same `/tmp/altairsim{n}`. A test reaches the slave side through `platform::openPtyPeer`, because the platform lint covers `tests/` too.

`runMcp` pumps while it waits, but only with a mirror. Between requests the worker used to block in `cv.wait`, and nothing called `pump()`: the tail of a long listing stayed in the mirror's queue until the next `run`. With `--mirror` it now wakes every 10 ms and calls `m.pump()` on the worker thread. Without a mirror the wait is unchanged.

Installing the resolver — in both mains

The grammar travels to a board as a function it is handed, never as knowledge it holds:

```
static void setResolver(EndpointResolver r);   // on the board (or the Sio2Port section)
```

Install it **once in each composition root**: `src/main.cpp` (the CLI) *and* `tests/main.cpp` (the test binary). Miss the test main and every unit test that connects an endpoint gets a board whose resolver is null. A

card that embeds a `Sio2Port` inherits the section's one resolver and needs no `setResolver` of its own.

The modem pins go both ways, and the chip owns polarity

```
struct LineStatus  { bool carrier=true, cts=true, dsr=true; bool ring=false; };  // far end -> card
struct LineControl { bool rts=false, dtr=false, brk=false; };                    // card -> far end
```

One rule governs every pin on every stream:

> **`true` is always "asserted".**

The pin-level inversions are real — the 6850's carrier and CTS pins are `/DCD` and `/CTS`, active low — but that is a fact about *that chip's pins*, and it stays inside the chip that has them. A stream that reported "carrier = true, meaning the pin is high, meaning there **isn't** one" would be exporting one chip's polarity to every other card in the machine, and the 88-SIO would be wrong for free. The wire carries a level in true sense; the honoring chip decides what it means — the same division as `PHANTOM*`.

A level has **no memory**. The stream says "carrier is down" for as long as carrier is down; the *chip* does the latching (a 6850 holds its DCD flag after the pin returns and clears it only on a status-then-data read). Put the latch in the stream and every stream re-implements it slightly differently.

A console or a file has none of these pins in any real sense, so it **asserts them all** — which is exactly what strapping DCD and CTS to ground on the connector does, and what period installers did constantly. That, again, is why there is no "what if nothing is plugged in" branch anywhere.

The line rate is the card's, and there is only one

```
struct LineParams { long long baud=9600; int dataBits=8, stopBits=1; LineParity parity=None; };
virtual bool setParams(const LineParams&, std::string& err);
```

There is exactly one line rate in a serial card and it is the UART's clock — a jumper. When a unit is connected to a **real host serial port**, the *card programs the port*: `SET sio0:a BAUD=300` restraps the card and the host UART follows. Only a `serial:` endpoint honors `setParams`, and it is the only stream that may **refuse** — an FTDI cable that cannot do 76800 baud is a fact about the world, and the card says so out loud through `drainLog()` rather than run at the wrong speed in silence. A `false` return does not mean the connection failed: the card stays strapped to what it is strapped to and goes on pacing the guest at that rate, because that pacing is the half the guest can actually measure.

A second, independent baud rate on the endpoint was considered and struck. It could only ever configure a *mismatch*, and a 6850 strapped for 300 driving a terminal set to 9600 does not give you a fast link on

real hardware — it gives you garbage. Every stream but a real serial port ignores `setParams` entirely: a socket has no baud rate, and pretending it did would pace an emulation against a fiction.

One related flag closes the loop. `pacesItself()` returns true for a stream that carries its own cadence — a cassette releases bytes at a wall-clock rate the CPU's speed cannot drag. When it is true the UART must **not** also impose its emulated line-rate gate, or the two clocks fight (see `Uart1602::poll` and `host/tape.h`). Every other line has no cadence but the one the UART's baud gives it.

A chip is not a card, and a serial section is not one either

`theory.md` states the chip/card seam: a chip is modeled from its **data sheet**, a card from its **manual**, and the inverting buffers and interrupt-enable flip-flops that live between the chip's pins and the S-100 bus are the *card's*. Serial adds one more object in the middle.

A concrete UART is a part in `src/chips/`: `Mc6850` (both halves of the 2SIO), `Uart1602` (the COM2502 — the 88-SIO and the ACR), `Intel8251` (the SBC console). A chip knows nothing about S-100; it has a clock, some pins, and a `ByteStream`.

A board with no UART at all, where the guest moves each bit with an `OUT` and a delay loop, uses `BitSerialLine` (`src/chips/bitserial.h`). It models the UART at the far end of the wire. It turns the levels the guest writes into bytes, and the bytes from the stream into levels the guest reads. Its levels are the wire's (true = MARKING). Which way up the bit is on the bus is the board's: the CompuPro Disk 1 inverts it on port 3 D7.

But the *glue* that turns `IN 10h` into "read channel A's status register" — decode, the even→status/odd→data dispatch, the single card-owned Clock deadline, interrupt aggregation, connect/units/properties, SNAPSHOT — is needed by **every** card that carries a 6850, and copying it onto each is how a bug gets fixed on one card and stays wrong on the others. So it lives once, in **`Sio2Port`** (**`src/chips/sio2port.h`**), a reusable serial *section*:

```
struct ChannelDef { std::string name; uint8_t offset; };   // the 2SIO has {"a",0},{"b",2}
Sio2Port(std::vector<ChannelDef> channels, std::function<void()> onIntChanged);
```

`Sio2Port` is **not a `Board`** — no `type()`, no id, no bus. The owning card holds one as a member and forwards the bus and lifecycle calls to it (see `src/boards/mits-turnkey.cpp`, one channel, and the 2SIO, two). It cannot reach the card's protected `clock_` or `intChanged()`, so it is handed both at construction. Parameterize it by the channels the card has — one or four — and the whole serial mechanism comes with it.

TDRE is a deadline, not a flag

A UART does not know a character has finished going out because the guest asked — it knows because *time passed*. So a serial section sets a `Clock` deadline for the moment the byte clears the shift register, and `writable()`/TDRE reflects that deadline, not an instantaneous flag. `pump()` takes an arriving byte off the line; if the line has not yet had time to deliver it, the deadline covers the gap. This is the single most common thing a naive UART gets wrong — read §4.4.1 of DESIGN.md before you put any work inside `assertsInt()` (theory.md explains why the poll must never be doing the card's clocking).

Interrupts

The full interrupt model — `assertsInt()`/`assertsVi()` as pure levels, the wire-OR the bus keeps, `intChanged()` after any pin move, `irqJumperProperty()` for the `none|int|vi0..vi7` vocabulary — is in `theory.md` and is not serial-specific; a parallel card wires interrupts the same way. For a serial board, the only things to remember:

- Give the board its strap with `irqJumperProperty("interrupt", …, irq_)` and it gets the ten choices, the spelling, tab completion and the `SHOW BUS IRQ` line for free.
- Override `assertsInt()`/`assertsVi()` to report the *settled* pin from the chip's state — a character waiting with the jumper installed asserts, and keeps asserting until the guest reads the character. It is a level; there is no queue to lose.
- Call `intChanged()` wherever the pending flag can move: a register written, a byte taken off the line, a deadline coming due. A missing one hangs the guest forever waiting for an interrupt that already happened — and it presents as "*the emulator locks up sometimes*".

An embedded `Sio2Port` aggregates its channels' pins and drives the callback the card bound to its own `intChanged()`, so a card that forwards to a section gets this right by construction.

Two shapes of serial board

Everything above supports two ways to build a serial card, and knowing which you are writing tells you where the code goes.

The chip-backed card emulates a specific UART. It embeds a `Sio2Port` (or a bare chip), forwards bus/lifecycle/connect to it, and adds only the card-specific glue: the base-port jumper, the inverting buffers, the interrupt straps. The 88-2SIO, the Turnkey, the SBC are this. The manual tells you the polarity and the port map; the data sheet tells you the chip. **This is the right shape when you are modeling real silicon** — and the project rule stands: never give the hardware a behavior it never had to make software happy.

The strap-configurable card does not model a chip's register map at all. **`StrapSerialBoard`** (**`src/boards/strapserial.h`**) — the chip-less strap engine — holds a `std::unique_ptr<ByteStream>` per channel and *synthesizes* each status byte from the stream:

```
uint8_t s = 0;
if (ch.stream->readable() != ch.straps.inverterGate) s |= (1u << ch.straps.davBit);   // XOR = inverter-gate
        inversion
if (ch.stream->writable() != ch.straps.inverterGate) s |= (1u << ch.straps.tbmtBit);  // both bits, one knob
```

Two boards ride this engine. **`GsioBoard`** (**`src/boards/gsio.h`**, **`type() == "gsio"`**) is the Generic SIO, which carries **two independent serial channels** — units `a` and `b`, configured under `[board.unit.a]` / `[board.unit.b]`. It makes no claim to be a specific card; a documented board that needs programmable word length, parity, stop bits or framing/overrun status is a separate emulated board, not a strap on this one. **`PropIoBoard`** (**`src/boards/propio.h`**, **`type() == "propio"`**) is the S100Computers Console IO Board, a single channel; being single-channel, its straps surface as board-level properties and its one unit is named `serial`. The operator *describes* each channel with straps rather than picking a chip: which port is status/control, which port is data, which status bit is `dav` (data available) and which is `tbmt` (transmit buffer empty), and whether the **inverter gate** is engaged — one knob, because both status bits pass through the same inverting buffer and so always share a polarity. Control-port writes are accepted and ignored — there is no chip to program. To make common cards turnkey it ships **built-in profiles** in one table (`serialBuiltins()` — `sior1`, `sior0`, `tuart`, `imsai-sio2`, `compupro-if2`, `compupro-ss1`), each just a bundle of those straps and trivial to extend: add one struct and its name becomes a `profile` choice and appears in the generated docs. The default profile is **`sior1`** (MITS SIO Rev 1), what the SSM 8080 monitor expects on its console. These cards are **polled, with no interrupts** — a deliberate first phase, because without a working control/interrupt-enable register a strapped TX-empty interrupt would storm (TBMT is asserted at idle). This is the right shape when the goal is to *reach* an abstract serial interface some software expects, not to reproduce a particular board's silicon.

Adding an IO-4 profile

The chip-backed `io4` (`src/boards/io4.{h,cpp}`, the SSM IO-4) is worth its own note, because the cheapest useful change you can make to a serial board lives there: **teaching it a new host personality**. Unlike `gsio`'s strap presets — a `dav`/`tbmt` bit pair — an IO-4 **profile** is a richer bundle, because the real card's W1/W2 headers strap *six* status signals to any data bit, through a buffer of either polarity, with the status and data ports optionally swapped. A profile is that whole map:

```
struct Io4Profile {
    const char* name;
    const char* help;
    int         stat[Io4Board::kNumStat];  // Dav, Ror, Rpe, Rfe, Teoc, Tbmt; -1 = not jumpered
    bool        invert;                     // 74368 negative sense if true, 74367 positive if false
    bool        pr;                         // S1/S2-PR: swap the status/data addresses
};
```

They live in one table, `io4Profiles()` (anonymous namespace in `io4.cpp`), transcribed from the IO-4 manual's applications section. To add a personality, **add a row** — for example the Rev-1 Altair console the SSM 8080 monitor wants:

```
{"altair-rev1",
 "Altair 88-SIO Rev 1 / the SSM 8080 monitor console: DAV=D0, TBMT=D7, status inverted",
 {/*Dav*/ 0, /*Ror*/ -1, /*Rpe*/ -1, /*Rfe*/ -1, /*Teoc*/ -1, /*Tbmt*/ 7}, true, false},
```

and its `name` becomes a `profile` choice, appears in `SHOW`, tab completion and the generated board reference, and needs no other edit. `applyProfile()` copies the row onto a channel's live straps; `profileName()` runs the comparison the other way — it reports whichever profile the current straps *match*, or `custom` if none does, so there is no stored profile field to drift out of sync with a hand-tweaked strap. That last point is the discipline: **the straps are the truth, the profile name is a read-back.** A row is only ever a preset, never state.

Two things to respect when writing a row. Strap an **error flag** (ROR/RPE/RFE) if the host's status layout has one — `i8251` does — even though the emulated UART reports it always-inactive: a driver that masks the bit still sees the right byte shape, and inventing line-noise to make it toggle is exactly the behavior the hardware never had. And `pr: true` **swaps the two port addresses** for that channel (data first, status last) — `imsai` needs it — which the addressing code reads back out of the same strap, so a profile and a hand-set `port_reversal` cannot disagree.

How to add a serial board

The general playbook is `adding-a-board.md`; the serial-specific steps on top of it:

1. **Own a stream, never null.** Hold a `std::unique_ptr<ByteStream>`, bind it to a `NullStream` in the constructor. Or embed a `Sio2Port` and let it hold the streams.
2. **`readable()` →RDRF, `writable()` →TDRE.** Synthesize your status byte from those; apply your card's polarity here, in true sense at the pins.
3. **Expose one serial unit** per channel (`units()` → `UnitKind::Serial`), wire `connect` / `disconnect` / `unitStream`, and route `pump()` to the stream.
4. **Install the resolver in both mains** — `src/main.cpp` and `tests/main.cpp` — via your board's (or `Sio2Port`'s) `setResolver`.

5. **If you touch a real serial port**, push `LineParams` through `setParams` on connect and on any baud change, and surface a refusal through `drainLog()`.
6. **If you model real silicon**, put the chip in `src/chips/` from its data sheet, keep the inverting buffers and interrupt-enable bits on the card, and reuse `Sio2Port` if it is a 6850. If instead you want a strap-configured abstract interface, `StrapSerialBoard` already exists (behind `gsio` and `propio`) — add a profile, do not write a new board.
7. **Test with a `ScriptedStream`** bound through the real `connect()` path: `feed()` bytes at the card, assert on `out()`, and check the status bits land where the straps put them.

Then regenerate the board reference (`cmake --build build --target docs-reference`) so the generated `ref/boards.md` picks up your properties, and add the board to the changelog and the prose board chapter — neither is test-enforced, so both drift silently.

The built-in terminal

`CONNECT sio0:a terminal` gives a serial line a windowed VT100 (or ADM-3A, VT52, H19) that the simulator draws itself — no telnet client, no external emulator. This chapter is how that is built, and how to add an emulation.

It is a **serial endpoint**, so read Serial I/O first: the terminal is one more `ByteStream` behind `resolveEndpoint`, and no board knows it exists. What is new is that the stream has a **screen** and a **keyboard**, and both live in the host `Display`.

Two halves: the screen engine, and the dialect

The terminal engine is `src/host/terminal/`, and it is deliberately split so that the part that differs between terminals is small:

Piece	File	Owns
`TerminalScreen`	`screen.h`	the attributed character grid and its primitive ops (`putGlyph`, `lineFeed`, `eraseToEol`, cursor). Sized `(rows, cols)` at construction — nothing is baked to 80×24.
`TerminalEmulator`	`emulator.h`	the **dialect**: a byte→ops state machine (`feed()`), plus the reply FIFO and key encoding. This is the only piece that changes per terminal.
`TerminalRenderer`	`renderer.h`	paints a `TerminalScreen` into a host `Display`, once per frame, using a `TerminalFont`.
`TerminalFont`	`font.h`	the glyph source. The bundled default is the authentic **DEC VT220** face (`boards/terminal-vt220font.h`, a 10×20 cell); the VDB-8024 supplies its own CGEN PROM through the same seam.

This engine was extracted from the VDB-8024 board (issue #244, Task 1). The SD Systems VDB-8024 is a built-in terminal that happens to be reached through I/O ports and speaks its own firmware dialect; `Vdb8024Board` is now a client of this engine, with an `SdVdb16Emulator` (`src/boards/sd-vdb8024.cpp`) as its dialect. The generic `terminal:` endpoint is the same engine reached through a serial `ByteStream` with a pluggable dialect instead.

`TerminalFont::glyphRow()` returns a **`uint16_t`** scan line with **bit 15 the leftmost dot**, so a cell may be wider than eight dots — the VT220 face is 10 wide. An eight-dot font (the VDM-1, the VDB CGEN) MSB-aligns its byte into the top of the word, keeping bit 15 leftmost for every font. The built-in terminal's phosphor palette (green default, amber) is set on its `TerminalRenderer` by `TerminalStream::setPhosphor()`, driven by the `phosphor=` connect-string option.

The emulator contract

`TerminalEmulator` (`emulator.h`) is short and worth reading in full. The load-bearing part is that a terminal is **bidirectional**, and everything flowing toward the guest goes through one FIFO:

- `feed(byte, screen)` — apply one display byte; advance any multi-byte escape sequence. May enqueue a **reply** (a report the guest asked for, e.g. a VT100's `ESC[6n` cursor position).
- `keyAscii(byte)` — a host keystroke with an ASCII code. The base passes it straight through; an emulator overrides only to remap.
- `keySpecial(Key)` — a host key with **no** ASCII (`Key::{Up,Down,Left,Right,Home}`). This is the classic difference: an arrow is `ESC[A` on a VT100, `ESC A` on a VT52, `Ctrl-K` on an ADM-3A. The base sends nothing; an emulator with arrows overrides it.

Reports and keystrokes both land in the **reply FIFO**, drained with `takeReply()`. That is correct: to a UART, a report and a keystroke are both just characters arriving on the line, and the guest cannot — must not — tell them apart. `reset()` abandons any partial sequence and clears the FIFO, as an S-100 RESET does.

Registering an emulation

The dialects are a name→factory table, `emulations.{h,cpp}` — the one place that knows which terminals exist, mirroring `serialBuiltins()`. Adding a terminal is:

1. Write the `TerminalEmulator` subclass (e.g. `src/host/terminal/vt52.h`).
2. Add one row to the table in `emulations.cpp` (`{"vt52", "DEC VT52", &makeVt52}`).
3. Give it grid-assertion tests in `tests/test_terminal.cpp`.

The endpoint grammar, the `HELP CONNECT` gloss and the error messages all read this table, so nothing else changes. **The first row is the default** when a `terminal:` names no emulation (`vt100`). Names match case-insensitively, like a board id or a property.

The four shipped dialects: `vt100` / `ansi` (a working editor subset — CSI moves, ED/EL, SGR, `ESC[6n`, DECSC/DECRC, DECCKM arrow flip); `adm3a` (the dumb CP/M terminal — control-code moves, `ESC =` cursor addressing, **no reports**, because the hardware had none); `vt52`; and `h19`, which is a VT52 superset in Heath mode and **delegates to `Vt100Emulator`** in ANSI mode (`ESC <` enters it, `ESC[?2l` DECANM returns).

The stream and the endpoint

`TerminalStream` (`stream.{h,cpp}`) is the engine wearing a `ByteStream` face:

- `write()` — guest → screen: feed each byte to the emulator.
- `read()` — guest ← the emulator's reply FIFO (reports **and** the keystrokes routed to this line).
- `pump()` — paint one frame into the host `Display`, on the main thread.

The `Display` and `TerminalFont` are **injected once at the composition root** (`setDisplay` / `setFont` in `src/main.cpp`), the same way the boards' `Display` and the endpoint resolver are — a stream built deep inside `resolveEndpoint` cannot be handed them, and they are session-lifetime host resources it only borrows.

The `terminal:` scheme is parsed in `resolveEndpoint` (`src/host/endpoint.cpp`): grammar `terminal[?emulation=vt100&size=80x24]` (`size` also spelled `windowsize`), validated **before** the window check, so a typo is a grammar error rather than a mysterious refusal. Then capability: a terminal needs a window, so `TerminalStream::hasWindow()` gates it — `SdlDisplay` answers `isWindowed()` true, `NullDisplay` false, and a headless build refuses `terminal:` cleanly at `CONNECT` rather than opening a serial line nobody can see. SDL is optional throughout: absent it, the endpoint is simply unavailable, never a compile break.

The one keyboard, and where it goes

Here is the constraint that makes this bigger than the printing endpoint. Today there is exactly **one** `SdlDisplay g_display`, shared by every video board, with **one** global key sink. The model is "one window, one keyboard." A `terminal:` line renders in that window and must receive its keystrokes — but so does the monitor, and so does a VDM-1 if the machine has one.

The routing (issue #244, Task 4):

- `TerminalStream::keyTarget()` names which terminal line owns the keyboard. A `terminal:` claims it on construction — **last one wins** — and its destructor releases it *only if it is still the owner*. That ordering matters: a `CONFIG LOAD` constructs the replacement line (which claims the target) **before** destroying the old one, so a blind clear in the destructor would strand the new terminal (see the config-load-replaces rule in `DESIGN.md`).
- The composition root's key sink (`src/main.cpp`) routes regular keys to `keyTarget()->keyAscii()`, falling back to `Console::instance().inject()` when no terminal is present — so the monitor and the VDM/Sol keyboard behave exactly as before.
- Arrows/Home carry no ASCII, and the `Display` normally collapses them to a single Sol-20 *byte* before any sink sees them (`emitSpecialKey` in `src/host/display.h`) — which a terminal could never re-encode in its own dialect. So there is a second seam, `Display::SpecialKeySink`,

delivering the key **symbolically**; when set it takes precedence over the byte table, and the composition root routes it to `keyTarget()->keySpecial()`, falling back to injecting the table byte into the `Console`.

Deferred: independent windows with per-window keyboard routing. That is the real fix for two concurrent terminals — and for the keyboard contention that already exists between a VDM-1 and a terminal in one machine — and it is the last unbuilt piece of the plan. Until a two-terminal machine needs it, one host window drives whichever line claimed it last.

The `[terminal]` transform chain

`src/host/filter.h` argues at length that the console's fold-the-bytes transforms (`strip7out`, `upper`, `bsdel`, …) must **not** live on a serial line: a line also carries XMODEM, and a `strip7out` there corrupts binary silently. But that file names one exception in passing — *only the console transforms, because only the console has a human on the end of it, and every one of these transforms is a fact about a **terminal**.* The built-in terminal is exactly that: a terminal with a human on the end. So it carries the same chain, as the `[terminal]` config section.

It exists because of a concrete bug (issue #244). An even-parity monitor — the MITS Programming System II — computes parity **into bit 7** of every character it prints. A carriage return `0x0D` goes out as `0x8D`; a line feed `0x0A` is already even and stays `0x0A`. A `terminal:` line is raw 8-bit, so `Vt100Emulator::feed` sees `0x8D`, which is `>= 0x20`, and prints it as a glyph instead of homing the cursor — while the `0x0A` still line-feeds. The symptom is "Enter feeds a line but never returns." `strip7out` masks bit 7 and both bugs vanish at once.

Design points, all in `src/host/terminal/stream.{h,cpp}`:

- **Modeled on `[display]`, not on a board.** `TerminalStream::Settings` is a `static`, and `TerminalStream::properties()` publishes it through the one `Property` layer — so SET, SHOW, TOML load and tab-completion all pick it up with no new plumbing. Config binding is at `src/config/toml.cpp` (a `[terminal]` branch beside `[console]` / `[display]`), and the monitor SET/SHOW/`showTerminal` sites mirror `display`. Like `[console]`, it does **not** round-trip through `CONFIG SAVE`, and it is accepted on a headless build (the setting is about what the machine wants).
- **A live-read `static`, global.** One section for the machine, like `[console]`; `SET terminal strip7out=on` reaches the running window because `write()` / `keyAscii()` read the static each time. `BsMap` is reused from `host/filter.h` rather than re-declared.
- **Outbound folds in `write()`, inbound folds in `keyAscii()` — NOT `read()`.** This is the one non-obvious call. `read()` drains the emulator's reply FIFO, and that FIFO carries the cursor reports the guest asked for (`ESC[6n` → `ESC[r;cR`) as well as encoded keystrokes. Folding in `read()` would mangle a report — an H19 `ESC n` reply can contain a lowercase coordinate byte, which `upper` would corrupt. Keystrokes enter through `keyAscii()`, so that is where `upper`, `strip7in` and `bsdel` belong; reports never pass through it. `test_terminal.cpp` has a guard for exactly this.

- `cr = cr | crlf` is the terminal's one line-ending knob (the console spells the same idea `crlf=on/off`). Default `cr` passes the guest's CR through untouched; `crlf` appends an LF after each CR, applied *after* `strip7out` so it triggers on the real `0x0D`.

The run banner and a non-stdio console

A related fix rode along. `Monitor::runFrom` announced `(no console connected)` whenever the console was not the **stdio** one — so a machine consoled on a `terminal:` window or a `socket:` was mislabeled as having no console at all. The banner loop already computed `anyRemoteLine` (a live line that is not the host terminal); it now also captures that line's scheme and prints `(console on terminal)` / `(console on socket)`. The honest `(no console connected)` is kept for the genuinely bare backplane — the ROM-talking-to-a-disk case — where no serial line is live.

Testing

`tests/test_terminal.cpp` drives the engine directly through a `NullDisplay`: feed a dialect's escape sequences, assert the grid (`screen()`), and assert `read()` returns the expected reports and key encodings. It needs no window, because the display seam is only exercised in `main.cpp`. The `[terminal]` transforms have their own section there (including the report-not-folded guard). Run it with `./build/altair_tests terminal`; the CONNECT grammar, the `[terminal]` SET/SHOW surface and the run-banner label are in `./build/altair_tests cli`.

Soft-sector floppy controllers

Every soft-sector floppy card — the Tarbell #1011/#2022, the SD Systems VersaFloppy, and whatever WD177x-based card turns up next — is built on the same three layers, and the whole point of them is that a card only ever touches the top one and reuses the two below unchanged.

> **A board decodes its I/O ports and straps its chip. It does not know what a sector looks like, where a track's bytes live in the file, or how a format is parsed. Those belong to the chip and the drive, and they are shared.**

That one rule is why adding the Tarbell was mostly a port decode and a boot PROM, why the DD card is the SD card plus four small overrides, and why this chapter exists once instead of a copy in each board's source. (For the *serial* seam this mirrors, read `serial-io.md` first — the shape is identical.)

The three layers, top to bottom:

Layer	Who	Owns
The **card**	a `Board` (`src/core/board.h`, e.g. `boards/tarbell.h`)	port decode, register bits, the drive-select latch, the **density strap** (`DDEN`), the boot PROM, interrupt straps
The **chip**	`Wd17xx` (`src/chips/wd17xx.h`)	the register file and the Type I/II/III command FSM; `doubleDensity` **is** the `DDEN` pin, `dataRateBits` the media bit rate
The **drive**	`DiskImageDrive` (`src/boards/floppy-drive.h`), a `FloppyDrive` (`src/chips/floppy.h`) over a `DiskImage` (`src/host/disk.h`)	CHS ↔ file offsets, synthesized ID fields, the **format parse**

A card reaches down to the chip (`Wd17xx::attach`, the straps). It never reaches into the `DiskImage`; the drive is the only thing that touches it.

The chip: `Wd17xx` and the parts

`src/chips/wd17xx.h` is the FD177x family. Read its header — every comment is load-bearing, and the chip/drive split is spelled out there and in `src/chips/floppy.h` at length. The essentials for a board author:

- **The class is the part, the file is the family.** `Wd1771` is single density (FM); `Wd1791` is single *and* double density (FM/MFM), with a side-select pin and a one-bit record type. The base `Wd17xx` is the whole register file and command FSM; a part supplies only the four things that genuinely differ

(step-rate table, Read-Address side byte, record-type bits, and which data-address-mark a Write writes). Build the part the card has; do not `#ifdef`.

- **No drive select, no side select, no motor.** The chip talks to ONE `FloppyDrive`; the card's select latch points it with `attach()`. Side is a card latch too (`setSide`).
- **`doubleDensity` is the `DDEN` pin, and `dataRateBits` is the media bit rate.** They are two values, and the board sets both from its control port. The rate is for byte timing and for the revolution byte budget (`trackImageBytes`). The density is what `Write Track` records a track at (`writeTrackImage`) and what the density check compares. **The rate does not give the density:** 8″ single density and 5.25″ double density are both 250 kbit/s. The chip is the **single source of truth** for both — do not duplicate either onto the drive.
- **The density check is a board strap** (`setDensityChecked`, off by default). On, an ID field recorded at the other density does not exist for the chip. See "The density model" below.
- **Wait-synced vs DRQ-polling.** A card whose data port stalls the CPU on a wait-state generator (PRDY) sets `setWaitSynced(true)`; then every command completes on the register access that would have stalled, one byte per access, and Lost Data is correctly unreachable. Both Tarbell boards are wait-synced. A DRQ-polling card leaves it off and gets byte timing. **A Type I command (Restore, Seek, Step) does not step in the cycle that loads it.** The step pulses go to the drive that is selected when they happen, and a driver may select another drive straight after it loads the command — the Tarbell CBIOS `HOME` does. So the stepping waits for the guest's next access to the board (`Wd17xx::touch()`), or for one step time if the guest never comes back; a plain `poll()` from the board's timer or the run loop leaves it alone. The chip's own register accesses call `touch()`. **The board must call it for every other port it decodes** — the drive-select latch (after the latch is applied), and the track and sector registers — guarded by `stepPending()`. A real part may get its first pulse out before the select lands; the model gives the new drive all of them. The one deadline a wait-synced chip keeps is the index pulse that ends a `Write Track`. See "The `trackImageBytes(rate)` budget" below.

The drive: `DiskImageDrive` over `DiskImage`

`src/boards/floppy-drive.h` is the generic adapter between the chip's pins and a flat logical `DiskImage`. A raw `.DSK` holds **sector payloads only** — no gaps, no address marks, no CRCs (`src/host/disk.h`). So:

- **ID fields are synthesized** from the mounted image's declared per-track `TrackFormat` (`sectorIdAt`): the track is the physical head position, the sector counts from the format's `startSector` (1 on a soft-sector card), and the CRCs always check (an image carries no rot).
- **`DiskImage` is CHS with per-track geometry.** Each `(track, head)` slot carries a `TrackFormat{density, sectors, sectorSize, startSector}`; offsets are a **running sum** over the slots (`rebuild()`), so a disk whose tracks differ in size — the mixed-density disk — is expressible where one global geometry could not say it.
- **The head position lives in the drive, not the chip's Track Register.** They may disagree; that is what a verify catches (Seek Error).

Geometry in real time — the FORMAT path

This is the reusable core. A soft-sector disk's geometry is **not** fixed at mount: sector size, count and density can vary track to track, and none of it is in the `.DSK`. So:

> **`Write Track` is the only command that establishes or mutates a track's geometry.** Reads and writes never do; they *validate* against what a track records.

The chip side is already done (`Wd17xx`, no per-board work): on `Write Track` the chip asks the drive for `trackImageBytes(dataRateBits)`, and if it is positive it enters the write phase, accumulates every guest byte into an internal buffer, and hands the whole revolution to `drive->writeTrackImage(buf, doubleDensity)` at the end. The first call carries the chip's data rate and the second its density — the chip is the single source of truth for both, and the drive keeps no copy: it derives the revolution byte budget from the rate and records the density. When `trackImageBytes(rate)` is `0` the chip sets **WRITE FAULT (S5)** instead — the honest answer for an empty drive or a controller that does not format.

`DiskImageDrive::writeTrackImage` (in `floppy-drive.cpp`) is the format FSM, run over the whole collected buffer:

```
gap … 0xFC(index) … gap … 0xFE track side sector N 0xF7 … gap … 0xFB <data…> 0xF7 … gap … (repeat)
        ^ID address mark  ^length code              ^data address mark   ^CRC-generate
```

Per sector: `sectorSize = 128 << N`, capture the first sector number as `startSector`, and **accumulate the data field until `0xF7`** — the CRC-generate byte, which can never appear as literal track data, so accumulate-until-`0xF7` is unambiguous (the `0xE5` fill and the `0xDD` density signature are ordinary data, not special). Then:

1. Derive `TrackFormat{density = (rate ≥ 500 kbit/s ? DD : SD), sectors, sectorSize, startSector}` and call `img->setTrackFormat(head, side, tf)`. That marks the slot valid and re-runs `rebuild()`, so the following tracks' offsets — and the growth cap — follow.
2. Write each sector's payload by a **sequential 1..N counter** from `startSector`, ignoring the header's possibly-skewed sector number, so the fill stays contiguous and the file grows in order. **Write exactly the bytes the guest streamed — never fabricate or pad the fill.** A data field shorter than the recorded `sectorSize` is a malformed track and `writeSector` rejects it → WRITE FAULT, which is honest.

Return `false` (→ WRITE FAULT) only if nothing parsed or a write could not land.

The ascending-track-order invariant

Correctness rests on one fact: **FORMAT writes tracks 0→N in order.** So when a track is (re)formatted to a *larger* geometry, `rebuild()` moving the following tracks' offsets clobbers nothing valid — they have not been written yet, or are about to be overwritten. Every real Tarbell format program formats ascending (`pd2/FORMAT.ASM`, `pd2/DFORMAT.ASM`). A fully-correct out-of-order / cross-density reformat of an *already populated* disk would have to shift the tail by the size delta first; that is **deferred** (see below).

Growth: `setExtendsOnWrite` and the dynamic cap

`DiskImage::setExtendsOnWrite(true)` lets the backing file grow as sectors are written, capped at `geometryBytes_`. For a soft-sector card that cap is **dynamic** — it rises as each track is formatted (`setTrackFormat` → `rebuild`). A recognized full disk never grows (its writes stay in bounds); a blank one grows track by track as it formats. The board turns it on at mount.

The density model

Density is one value with three faces, and they must agree:

Face	Where
The `DDEN` pin	`Wd17xx::doubleDensity`
The board I/O bit	e.g. Tarbell DD `OUT FC` bit 3 (`reference/Tarbell_Floppy_Disk_Interface_Manual.md`: "D3 = density"); Cromemco port 34 D6
The recorded per-track density	`TrackFormat.density`, written by `Write Track`

The board sets `doubleDensity` from its control-port bit; the chip hands it to the drive's `Write Track` commit; the drive **records** it into `TrackFormat`. The data rate (`Wd17xx::dataRateBits`) is a separate value. On the Tarbell DD and the VersaFloppy it follows the density bit (250 or 500 kbit/s). On the Cromemco boards it is MAXI × DDEN: only 8″ double density is 500 kbit/s, and a 5.25″ double-density track is recorded at 250 kbit/s. So a double-density card formats a mixed disk — SD track 0, DD tracks 1-76 — from the guest's per-track `OUT FC` density bit, and it also *reads* plain single-density media. The guest-side proof of the board bit driving the chip is `pd2/DFORMAT.ASM` (`ORI 8` / `OUT FC` before a DD format); cross-reference the WD `WD177X-00` datasheet's `DDEN` description.

Reads and writes **validate geometry** — the addressed track's recorded sector layout via `locate()` — and return **Record Not Found (S4)** on an unformatted / out-of-range track, never WRITE FAULT. The **format path records density; it is never density-gated.**

The read-side density check

A real part that is clocked for FM cannot see an MFM address mark, and the reverse. The chip models this with a board strap, `Wd17xx::setDensityChecked(bool)`, which is **off by default**.

- **On:** the drive reports how each ID field is recorded (`SectorId::doubleDensity`, from `TrackFormat.density`). The chip passes by an ID field whose density is not its own (`doubleDensity`, the same value `Write Track` records a track with). `Read Address` and the Type II search end in **Record Not Found**, and the Type I verify ends in **Seek Error**. A write to a sector at the wrong density is Record Not Found too.
- **Off:** the medium decides. A track formatted DD reads back with the controller strapped SD.

Board	Strap	Why
VersaFloppy I / II	**on**	The DDBIOS finds the disk type by trying each density until `Read Address` succeeds (`DDB200.ASM`, `USL1`). Issue #691.
Tarbell DD	**on**	The `OUT FC` density bit must match the track. Issue #692.
Cromemco 16FDC / 64FDC	**on**	Port 34 D6 must match the track. Issue #692.

The check compares densities, not rates. The Cromemco boards run 5.25″ double density at 250 kbit/s, the 8″ single-density rate; `acceptance-cdos-5in` boots such a disk.

Mount vs. format — where geometry starts

`describeGeometry` (the board's size probe) establishes the *initial* geometry so a recognized disk is usable immediately with no FORMAT:

- A recognized size → that format (padding-tolerant via `sizeMatches`, for the XMODEM pad).
- **A blank / short image → an *unformatted* disk**, mounted at the card's track count with **empty** per-track geometry (no ranges): READY and steppable, every access RNFs until `Write Track` lays a track down. This is what makes `MOUNT … CREATE` (a 0-byte file) formattable. Empty-pending is preferred over fabricating slots, since `Write Track` is the sole source of geometry. The default rule matches SIMH's `tarbell_attach`: *anything that is not a recognized size is single density.*
- Oversized / garbage is still an error — a real track is never larger than one revolution.

A dual-density controller's probe is a **superset**, not a single-size gate: the Tarbell #2022 recognizes its mixed disk (499,456), a plain SD disk (256,256, read as all single density), and a blank (unformatted, formattable) — because the DD controller genuinely reads SD media too.

The `trackImageBytes(rate)` budget — the load-bearing number

Under wait-synced operation `Write Track` completes when the collected buffer reaches `trackImageBytes(rate)`. So that value **is** the per-track raw byte budget, and it is the one number most likely to bite:

- **Too small → the last sectors truncate**, because the command commits before the guest has streamed them.
- **Too large → a guest that pads until the command ends sends more gap than a real track holds.** The format still works, but the number is wrong.

A guest can also send less than the budget. Some format programs send a counted track and then read status until the chip ends the command at the index pulse. The SD Systems `FORMAT.COM` for CP/M Plus sends 9,672 bytes for an 8″ double-density track of 10,416. The wait states hold the CPU, not the disk, so the index pulse still comes. When a DRQ of a `Write Track` has had no answer for one revolution of machine time (`trackImageBytes` byte times), the chip fills the rest of the track with zeros, sets Lost Data and commits the track. That is what the byte-timed model does one byte at a time. If the DRQ with no answer is the first one, the command ends with Lost Data and writes nothing, as the data sheet says. The deadline starts again at each DRQ, so a guest that continues to send bytes never reaches it. `Wd17xx::nextEdge` reports the deadline, so the command also ends for a guest that waits for INTRQ. A sector write has no such deadline (issue #693).

It is derived from the chip's data rate **and the drive's rotation speed** — one revolution is `rate / (8 × rev/s)` bytes (`rate/8` bytes per second ÷ the revolutions per second). An 8″ drive turns at 360 RPM = 6 rev/s: **5208** at 250 kbit/s (8″ SD) and **10416** at 500 kbit/s (8″ DD). A 5.25″ mini turns at **300 RPM = 5 rev/s**, a longer revolution — `rate / 40`: **6250** (5.25″ SD) and **12500** (5.25″ DD). The RPM is a physical property of the drive, so the card sets it per mounted drive from the diskette's size (`DiskImageDrive::setRevsPerSecond`, default 6 — a card that never sets it, like the Tarbell, is unchanged); the chip stays the single source of the rate.

The budget must be `≥` everything the format program streams before its trailing gap. `pd2/FORMAT.ASM` streams ~4882 structured SD bytes then pads with `0xFF` (its `ENDTRK` loop) until the controller signals INTRQ — which is exactly when the buffer hits the budget; `pd2/DFORMAT.ASM` streams-until-INTRQ the same way for each DD track (~10114 structured bytes, comfortably under 10416). Because the rate is the chip's, the DD card gets 5208 for SD track 0 and 10416 for the DD tracks from one mechanism, and the VersaFloppy's 5.25″ formats get 6250/12500 from the same one. Validate this number against the format program's gap tables, not by "it booted."

The other chip: the µPD765A

The CompuPro Disk 1 and Disk 1A do not use a WD part. The two boards share one class, `CompuProDiskBoard` (`src/boards/compupro-disk.{h,cpp}`), and each board is a small class over it (`compupro-disk1.cpp`, `compupro-disk1a.cpp`). Their chip is an NEC µPD765A, `Upd765` in `src/chips/upd765.{h,cpp}`, modeled from the data sheet in `reference/NEC uPD765A FDC.md`. The chip uses the same drive layer: `FloppyDrive` and `SectorId` are in `src/chips/floppy.h`, and the board puts a `DiskImageDrive` in each slot. The three layers of this chapter hold, but four things differ from the WD chip:

- **The chip selects the drive.** The 765 drives the US0/US1/HD pins itself, so it reaches its drives through `Upd765::Wiring`, a set of callbacks, and not through `attach()`. The board decides what the pins reach. The Disk 1A uses the callbacks to put its Drive Select Register between the chip and the cable.
- **The density check is in the chip.** The MF bit of each command is checked against the recorded density of the track. A read at the wrong density finds no ID field and ends with Missing Address Mark (ST1 MA). No board strap turns the check on.
- **Data moves by DRQ and DACK.** The board answers DRQ with `dackRead()` or `dackWrite()`, one byte for each DMA grant. Non-DMA mode (ND = 1) is also built. The 765 has no wait-synced mode and does not model byte timing, so it never reports an overrun.
- **The clock input scales the timing.** `fdcClockHz` is 8 MHz for an 8″ drive and 4 MHz for a 5.25″ drive, and every Specify interval doubles at 4 MHz. The Disk 1A sets it from bit 5 of its Drive Select Register. The Disk 1 runs 8″ drives only, so it is always 8 MHz.

Format a Track takes C, H, R and N for each sector through the data path. The chip gives them to `FloppyDrive::formatTrack()`, a separate entry point from the `writeTrackImage()` of the WD chip. Any other board with a 765 or an 8272 can use `Upd765` as it is.

The flat-`.DSK` limitation

A raw `.DSK` cannot record varied per-track geometry — it is payload bytes only. So the geometry is **re-derived from the file size on every remount** (`describeGeometry`), which is fine for a uniform SSSD disk and the one standard mixed disk, but a `.DSK` that had been formatted to some *custom* per-track layout would lose it across a remount. An IMD/TD0-style container that carries its own sector map would fix this — and is explicitly never coming (DESIGN.md §7.3): such files are converted to raw beforehand.

Deferred

- **Shift-tail resize** — on a `Write Track` that changes a *populated* track's total size, `memmove` the following data by the size delta before `setTrackFormat`, so a partial / out-of-order / cross-density reformat stays correct. Not needed for blank format or a whole-disk ascending reformat.
- **Read-side density gate** (above): reads validate geometry, not density. Deferred until a workload needs it.
- **A real-CP/M FORMAT/DFORMAT acceptance test.** *Both* densities are now covered end-to-end at the **board level** in `tests/test_tarbell.cpp` — the real `FC`/`FB` port discipline, all 77 tracks, the file growing (256,256 SSSD on the #1011; **499,456 mixed** on the #2022, SD track 0 then 76 DD tracks driven by the per-track `OUT FC` density bit), and the `0xE5` fill reading back at the right per-track geometry. A guest-driven `DFORMAT.COM` run on the tracked DD master is feasible (the master is `tests/media/tarbell/TARBELLDD-CPM22-SSDD-48K.DSK`) but the 9600-baud console makes the interactive prompt automation slow and stale-buffer-prone, so the guaranteed proof stays the board test.

Adding another soft-sector controller — the checklist

1. **Build the right WD part** in the board's `buildChip()` — `Wd1771` for a single-density card, `Wd1791` for one that does double density — and `setWaitSynced(true)` if the data port stalls the CPU.
2. **Strap density** from the control-port bit into `chip_->doubleDensity`, and set `chip_->dataRateBits` to the rate the board clocks the part at (leave both at their defaults, single density at 250 kbit/s, for an SD-only card).
3. **Size-probe with a blank fallback** in `describeGeometry`: recognized sizes → their format; anything smaller → empty per-track geometry (unformatted); oversized → error.
4. **`setExtendsOnWrite(true)`** on the image at mount, and enable formatting on the drive (`setFormatting(true)`) — leave it off (the default) on a card that does not format, and `Write Track` keeps faulting. Neither the byte budget nor the density is passed here: both are the chip's, derived per call from the data rate it hands the drive.
5. **Reuse `DiskImageDrive`'s format path** unchanged — the parse, the sequential fill and the `setTrackFormat` / `rebuild` are controller-agnostic.
6. **Set the drive's RPM** (`setRevsPerSecond`) at mount if it holds anything other than 8″ media — the budget denominator. Skip it for an 8″-only card (the default 6 is correct).

Two worked examples

- **Tarbell #1011/#2022** (`src/boards/tarbell.cpp`) was the first: an SD card and a DD card sharing `describeGeometry`, the DD one probing a *superset* of sizes (mixed, plain-SD, blank). All 8″, so it never touches the RPM knob.

- **SD Systems VersaFloppy** (`src/boards/sd-versafloppy.cpp`) was the second, and exercised the parts the Tarbell did not: **all ten** SD Systems formats — five geometries × single/double sided — including **256-byte** double-density sectors (the length code flows straight through the parser) and **5.25″** media (step 6, `setRevsPerSecond(5)`). Two lessons it added:
  - **The size collision.** `8sd-ds` (f1) and `8dd256` (fC) are both 512,512 bytes. The probe resolves it by *order* — the format table lists `8dd256` first, so an unforced probe of 512,512 lands on the SDOS master; `media=8sd-ds` forces the other. When two formats share a byte count, list the default first and require `media=` for the other.
  - **Double-sided blank-grow rides the ascending-slot-order invariant.** A blank double-sided disk grows contiguously only if the guest's format order matches the image's slot layout, so `setExtendsOnWrite` never has to fill a gap. The VersaFloppy lays disks **cylinder-major** (interleaved: `T0H0, T0H1, T1H0, …`, per the vf.z80 dumps), and the DDBIOS `FMAT` records **both sides of a cylinder before stepping** (`DDB200.ASM` `NXTRK`) — the two agree, so it is safe. A card whose format program wrote all of one side before the other under this same layout would leave gaps and need the shift-tail resize (still deferred).

Banked memory — the model, and where it was wrong

> **Status: the fix has landed.** Banking is now its own board, **`bankmem`** (`docs/boards/bankmem.md`, `src/boards/bankmem.{h,cpp}`), with one decoder per real card; the `memory` board is plain, unbanked RAM/ROM again. The `b810` type is gone and the SIMH-derived `bank_type` table is deleted. This page is kept as the **audit that led there** — it records what the manuals say, what the old model said, and why the two diverged. The *decisions* taken are recorded at the end (*The fix that landed*).

This page is a developer-facing audit of how `altairsim` modelled S-100 memory-bank switching (`src/boards/s100-memory.h` `BankType` / `BankSpec`, `src/boards/s100-memory.cpp` `kBanks[]`) against the **period manuals** for the real cards, distilled in `reference/`. It exists because that audit turned up a problem worth writing down before it is fixed: **most of the banked encodings we ship do not match the hardware, and the reason is that they were taken from another emulator's source instead of a manual.**

Nothing here changes code. It records what the manuals say, what our model says, and where the two diverge, so the correction (a separate branch — see *The follow-on*) starts from facts. The per-card facts below are sourced to the five new references:

- `reference/SD Systems ExpandoRAM.md` — the original ("I")
- `reference/SD Systems ExpandoRAM II.md`
- `reference/Vector Graphic 64K Dynamic RAM.md`
- `reference/North Star HRAM.md`
- `reference/Cromemco 64KZ RAM.md` (covers 64KZ and 64KZ-II)

The cardinal rule we broke

`docs/sources.md` states the rule at the top: "**Period manuals and first-hand artifacts only. Never read another emulator's source to learn how hardware works — that explicitly includes SIMH / AltairZ80. Second-hand facts inherit second-hand mistakes.**"

The banked-memory encodings in `kBanks[]` were taken from Patrick's SIMH `s100_bram.c`, not from the period manuals — and it shows. Of the four banked types we can now check against a manual, **only `vram` (Vector Graphic) is right.** `eram`, `cram`, and `hram` each diverge from the real card, in exactly the way the rule warns about: `s100_bram.c` is a *generalization* (`{port, banks, one-hot?, mask}` swapping a whole 64K plane), and the real cards are not generalizations — each is a different mechanism, and three of them

do not fit that shape at all. The comment block in `s100-memory.cpp:14` and the prose in `docs/boards/s100-memory.md:213` ("*five real cards, and no two alike*") present the SIMH table as if it were the hardware. It is not. This is a genuine violation of the project's central sourcing discipline, discovered only when the actual manuals were read.

The scorecard

`bank_type`	Real card (manual)	Real scheme	Our model (`kBanks[]`)	Verdict
`eram`	SD Systems ExpandoRAM **I**	**No I/O port at all** — flat memory-mapped; a DIP switch straps 4 chip-groups to fixed 8K/16K address slices	port **FF**, 8 banks, binary (byte = bank), swap 64K plane	✗ **wrong**
`eram`	SD Systems ExpandoRAM **II**	port **FF**, byte = **page number** (0–9); an 82S130 PROM + octal board-select switches decode it to a 32K/48K **partition**; 4 chip-banks/board	(same `eram` row)	✗ **wrong** (port matches, encoding does not)
`vram`	Vector Graphic 64K	port **40**, **one-hot** `1<<bank`, 8 banks, reset → bank 0	port 40, 8 banks, one-hot	✓ **correct**
`cram`	Cromemco 64KZ / 64KZ-II	port **40**, byte = **8-bit mask of active banks** (bit N ⇒ bank N on/off, *several at once*), 8 banks	port 40, **7** banks, one-hot **select-one**, mask `0x7F`	✗ **wrong**
`hram`	North Star HRAM	port **C0**, bit 0 = on(0)/off(1), bits 1–7 = **one-hot bank toggle**, ≤ 6 usable banks, old-off-then-new-on	port C0, **16** banks, binary, mask `0x0F`, swap 64K plane	✗ **wrong**
`b810`	*(claims "AB Digital Design B810")*	**unsourced** — no manual; cannot be verified	port 40, 16 banks, binary	✗ **remove**

`eram` — matches neither ExpandoRAM

The `eram` byte-is-the-bank / 64K-plane-swap model matches **neither** real ExpandoRAM. The original (I) has *no I/O port whatsoever* — its J1 pinout carries only memory-bus signals, and its "banks" are four fixed chip-groups strapped to address slices by a DIP switch; it is an ordinary static-address memory board, not a bank switcher. The II *does* use port FF (so our port is right), but the byte it takes is a **page number** decoded by an on-board 82S130 PROM into a 32K or 48K partition, with the board's identity set by octal board-select switches — not a bank number, and not a 64K plane. Our `eram` is the SIMH generalization sitting on the II's port.

`vram` — correct, and worth keeping as the proof

Vector Graphic is exactly what we model: port 40H, one-hot `1<<bank`, 8 banks, and bank 0 force-enabled at reset (the manual's theory-of-operation even explains the latch-clear + DO0 inversion that makes reset electrically equal to writing `0x01`). The **OASIS quirk** (`0x41`/`0x42` tolerated as banks 0/1) is empirical, not in this manual, and stays empirical. This is the one banked type whose encoding an implementer can trust today.

`cram` — wrong mechanism, not just wrong numbers

Cromemco's BANK SELECT byte is a **bit-mask**: "*BIT 0 (LSB) controls BANK 0, BIT 1 controls BANK 1 … a logic 1 control word bit activates its corresponding memory BANK; a logic 0 … deactivates*" (64KZ manual p. 17). Several banks can be active at once — `OUT 40H,28H` activates banks 3 **and** 5. We model a **one-hot select-one** with only 7 banks and bit 7 masked off. That is wrong on the bank count (8, not 7), on the masking (bit 7 *is* bank 7), and on the core semantics (mask-enable-many vs select-one). The "seven banks / bit 7 is not a bank" story in the code and docs is a SIMH artifact, not Cromemco hardware.

`hram` — the byte is not a bank number

North Star's port-C0 byte is **bit 0 = on/off command, bits 1–7 = a one-hot address of which bank the command acts on** — banks are toggled individually (software must switch the old bank off before the new one on), and one of bits 5/6/7 is spent on parity, leaving ≤ 6 usable banks. We model a 16-bank binary plane-swap. Verified directly against the scan (`MVI A,08H` = "turn on bank 3", `MVI A,09H` = "turn off bank 3"). Wrong on encoding, bank count, and protocol.

`b810` — unsourced, remove it

`b810` ("AB Digital Design B810") has no manual behind it and cannot be verified (§0.1 forbids shipping hardware behavior we cannot source). Patrick's decision: **remove it.** It is also the "fifth card" that props up the "*five real cards*" and "*three of the five share port 0x40*" narrative in the code and docs — remove it and that narrative must be re-counted (the real port-0x40 sharers among the sourced cards are `vram` and `cram`).

The fix that landed

The correction became the `bankmem` board (`docs/boards/bankmem.md`). What was done, and the design decisions taken:

- **Banking is its own board, `bankmem`, with a `card=` variant strap** — one class, four per-card decoders (`vector` one-hot select-one, `cromemco64kz` 8-bit mask, `northstar` on/off + one-hot

toggle, `expandoram2` PROM page-select). This was chosen over piling more knobs onto `memory`: the real cards do not share one parameterization, and `DESIGN.md` §4.3 argues a board must own its decode. The unifying model is *segments* (a RAM slice + address window + enabled flag); the port write toggles the flags per the card's rule.

- **`memory` is plain RAM/ROM again** — no `bank_type`, no `banks`/`bank` properties, no I/O port, a flat 64K store. This is also the honest model of the ExpandoRAM I (no port at all).
- **`b810` removed** — unsourced (§0.1).
- **`eram` retired** — the ExpandoRAM I is plain memory; the ExpandoRAM II is `card=expandoram2`.
- **`expandoram2` is a documented approximation** — a binary page-select, because the real 82S130 PROM decode is not transcribable from the scan. The caveat is stated in the board doc and in the code; a PROM dump would let the *page decode* be made faithful. **What the PROM's stock variants do is modelled, though** (follow-up, `feat/bankmem-partition-ram`): `partition=ex32|ex48` gives a fixed **common region** shared by every page (EX-32 = 32K banked + 32K common; EX-48 = 48K + 16K), and `ram=` sets board size up to 256K. That is what a banked CP/M needs — the resident OS and the bank-switch routine live in common and survive the page `OUT`. SD Systems' own banked CP/M 3 does exactly this (bank number → port FF, `COMBAS=C0` ⇒ `partition=ex48`); the general mechanism is now in place, while board-select across multiple coordinated boards remains the approximated part.
- **`v2z80rom` was kept as its own board** — its onboard paged **ROM** overlay (two 4K EEPROM pages at F000-FFFF, phantom-shadowing RAM, tied to a CPU card's identity) is a different thing from an S-100 banked-RAM card; folding it into `bankmem` (a RAM board) would reintroduce the very "ROM on a banked card is unsourced" tension the design refuses. Closed as: stays separate.

See also

- `docs/boards/s100-memory.md` — the board's design doc, now describing plain RAM/ROM only.
- `DESIGN.md` §10.2 — why there is no `BANK=` in the monitor.
- `docs/sources.md` — the sourcing rule this page documents breaking.

The PS II monitor loader, inside

The MITS **Programming System II** — Processor Technology's editor, assembler, monitor and debugger for the Altair — boots from cassette in a way that repays a close read, because the mechanism it uses to bring in its monitor is the same mechanism the rest of the package uses to bring in everything else. Booting the monitor is **two stacked loaders**: a tiny hand-toggled bootstrap that installs a block-load interpreter, and that interpreter reading the monitor itself. Once you see the second one you have the whole Software Package II — the editor, the assemblers and the debugger are nothing but data for it.

> **The payoff of digging through the bootstrap is the block-load protocol underneath it.** The bootstrap is a one-time trick to get that interpreter into memory when nothing yet can interpret anything. Everything after boot — including every tool you `EDT`, `ASM` or `DBG` from tape — is addressed, checksummed blocks fed to the same resident engine.

The source assets live in `tapes/MitsPS2/`: `LDRPS2.ASM` / `LDRPS2.HEX` (the toggle-in bootstrap), `PS2-MON.TAP` (the monitor tape), `PS2-EDT.BIN` / `PS2-ASM.BIN` / `PS2-AM2.BIN` / `PS2-DBG.BIN` (the tools), and the operator machine files `ps2.toml` and `ps2int.toml` (the second is the interrupts variant — an 88-VI/RTC and sense switch A9). Boot either with `altairsim ps2` / `altairsim ps2int`; to watch the loader run, drive it over `--mcp` and set a breakpoint (see `docs/manual/mcp.md`).

Layer 1 — the hand-toggled bootstrap

`LDRPS2.HEX` is the twenty bytes you would toggle in at `0x0000` on real hardware. It reads the 88-ACR one byte at a time and copies the second-stage loader **downward** into `0x0F00–0x0FAD`, then jumps to it. The whole thing is in `tapes/MitsPS2/LDRPS2.ASM`; the core is a five-instruction loop:

```
        lxi     h,0FAEh         ; H=0F (page), L=AE (=174 = leader byte = byte count)
loop    lxi     sp,stack        ; init SP so a RET jumps back to loop
        in      06h             ; ACR status
        rrc                     ; new byte ready?
        rc                      ;   no  -> RET -> loop
        in      07h             ; the byte
        cmp     l               ; is it the leader byte?
        rz                      ;   yes -> RET -> loop  (skip leader)
        dcr     l               ; not leader: step the address down
        mov     m,a             ; store it (reverse order)
        rnz                     ; more to go? -> RET -> loop
        pchl                    ; L hit 0 -> jump to 0F00, the code we just loaded
```

The trick worth naming is that **register `L` does triple duty**: it is the expected leader byte, the remaining byte count, and the low byte of the descending store address, all at once. That only works because the payload is exactly **174 bytes** and the tape's leader byte is `0xAE` — the same number. `H:L` starts at `0x0FAE`; each stored byte decrements `L`; when `L` reaches zero the last byte has landed at `0x0F00` and `PCHL` jumps there. One register, because the tape was authored so the three quantities coincide.

This is the **4K BASIC v3.2** bootstrap (`lxi h,0FAEh`), *not* the 8K one. The distinction is invisible from the tape: the leader byte `0xAE` is the low byte of the load address, so it is identical for the 8K loader (which would place the code a page higher) — nothing on the tape tells the two apart, and neither can you until you disassemble what landed. Load this second stage 4K too high and every one of its own jumps lands in the empty page below it and the machine wanders off silently. The `LDRPS2.ASM` header spells this ambiguity out at length; the `ps2.toml` / `ps2int.toml` comments repeat the warning where an operator meets it.

Layer 2 — the block-load protocol

The 174 bytes that land at `0x0F00` are the real loader, and it is general-purpose. It:

1. **Reads the sense switches** (`IN 0FFH`) to learn which serial port pair the console is on.
2. **Self-modifies its own `IN` instructions** — it patches the port operands of its own read instructions to the UART pair the switches selected, so one image serves whichever card is present. (This is why a linear disassembly of the loader as it sits on tape is only *half* the story: some of its bytes are rewritten before they run. See the next section for the sharpest instance.)
3. **Runs a command loop** over two commands read from the console/tape stream:
   - `'<'` — load a block: a length, a load address, that many data bytes, and a checksum. The block is range-checked, stored, **read back and verified**, and checksummed before the loader accepts it.
   - `'x'` — read an address and **execute** (jump to it).

`PS2-MON.TAP` after the bootstrap payload is exactly this: eleven `'<'` blocks, the low RST-vector block loaded **last**, and a final `x 0040` that warm-starts into the freshly assembled monitor. The monitor is not a single image with an entry point of its own — it is a handful of checksummed blocks and a jump, delivered by the interpreter the bootstrap just installed.

The `0x0F27` byte — filler *and* code

While reading the second stage under `DISASM`, one byte reads as nonsense: `0x0F27`. It looks like filler that only exists to make the loader assemble, and removing it "to clean up the disassembly" is exactly the wrong move — because **the byte is both filler and code, decoded two different ways depending on how control reaches it.**

The sense-switch setup ahead of it is a chain of conditional jumps, one per valid switch setting. Two paths run *through* this region:

Reached at	Bytes	Decodes as	When
`0F27`	`DA 06 06`	`JC 0606`	all the preceding conditional jumps fell through
`0F28`	`06 06`	`MVI B,06`	a preceding conditional jump landed on `0F28`

The single byte `DA` at `0F27` is either the opcode of a `JC` (when execution flows straight into it) or skipped entirely (when a jump lands one byte past it, on the `06 06` that then reads as `MVI B,06`). No linear disassembler can render this honestly: a disassembly assigns each byte to exactly one instruction, and this byte belongs to two. That is the real answer to "why does the loader disassemble into garbage here" — the garbage is inherent to an overlapping instruction, not a defect in the tool and not a region you could mark as "data" without lying the other way. Following it needs a breakpoint at the main-loop entry and a walk through each valid sense-switch path (`docs/manual/mcp.md`), not a static listing.

One theory that did *not* survive the digging: that the filler exists to shift the byte stream past a leader-match collision, or that the monitor reuses this second-stage loader to bring in its extensions. The reuse theory is disproved directly — loading the editor overwrites `0x0F00–0x0FAD`, where the second-stage loader sits, so it cannot still be resident to be reused. The extensions come in by a different door.

The tools are pure block-load data

That different door is the block-load interpreter, still resident from the monitor boot. The editor tape is the clean example. `PS2-EDT.BIN` (1920 bytes) has **no leader trick and no second-stage loader** — just an `EDT` name fragment, eight `'<'` blocks landing at `0x0A40–0x1150`, and a closing `x 0A40`. It is nothing but addressed, checksummed data for the engine the monitor already installed; live, an `EDT` load reads 1910 of the 1920 bytes (the last ten are trailing padding) and a dump at `0x0A40` matches the tape byte for byte.

This is the I/O-Table design the PS II manual describes. `EDT` loads a program of that name from wherever the symbolic name `ABS` points — `ABS` defaults to `TY` (the console), which is why `OPN ABS,AC` must first repoint `ABS` at the cassette driver before the load will come off tape. The same shape covers the rest of the package: `PS2-ASM.BIN`, `PS2-AM2.BIN` and `PS2-DBG.BIN` are all block-load data over the resident interpreter. Understand the monitor boot and you have understood how every one of them arrives.

Assembling a ROM (`.ASM` → `.HEX`)

When a built-in ROM arrives as period **source** rather than an image — an OCR'd monitor listing, a disassembly someone posted — you have to assemble it before it can go in `roms/<NAME>/`. Do it the way the machines did it: **run the period assembler inside the simulator.** The result is a `.HEX` (or `.PRN` listing) that came out of the real M80/L80, on the real 8080, not out of a modern cross-tool that might round a corner differently.

> **Never use `cpmtools` (`cpmcp` , `cpmls` , `mkfs.cpm`) to move files on or off a CP/M disk.** The simulator does that for you through the **host-bridge** board. `cpmtools` is a separate, unrelated toolchain that is easy to reach for and wrong here — it bypasses the machine, and the machine is the thing under test.

The setup

You need a CP/M machine with three things on it: the **assembler** (`M80.COM` , `L80.COM`), the **host-bridge utilities** (`R.COM` , `W.COM` , `HDIR.COM`), and room to work. `examples/cpm/cpm22b23-56k.dsk` carries all of them. For a large source or many modules, mount a fresh copy of that floppy as the tool drive and an **8 MB CP/M image** as the work drive — a floppy has only ~18 KB free after the tools.

A machine file for that, launched from the directory that holds your `.ASM` files (the host-bridge sandbox is the launch directory):

```
[machine]
name = "asm"
[[board]]
type = "8080"
id   = "cpu0"
[[board]]                       # host-bridge: R/W move files host <-> CP/M
type = "hostbridge"
id   = "hb0"
[[board]]
type = "gsio"                   # serial console: channel a at ports 0/1
id   = "io0"
  [board.unit.a]
  connect = "console"
# ... your disk controller + A: (8 MB work) and C: (tools floppy) ...
```

Drive it with `--mcp` , never a hand-rolled `expect` script (see the MCP chapter). MCP does not run the machine file's `startup>` , so boot yourself: `run {from:<boot PROM address>}` . **Stop each command on**

idle, not by matching the prompt — the CP/M prompt (`A0>`) recurs, and matching it splits the next line you send.

The recipe

M80 emits only a relocatable `.REL`; **L80** turns that into an Intel `.HEX`. Two things bite:

1. **M80's default source extension is `.MAC`.** Name the source explicitly: `M80 MON,MON=MON.ASM`, or M80 reports `?File not found` and drops to its `*` prompt (where your next command becomes garbage).
2. **The source must be relocatable.** `.8080` is fine; an `ASEG` with a high absolute `ORG` is not — L80 can't buffer it under CP/M and dies with `?Out of memory`. Strip the absolute top `ORG` and let L80's `/P` set the origin.

For one module at origin `F000`:

```
R MON.ASM MON.ASM T           ; host -> CP/M  (T = text: CRLF, trims trailing ^Z)
M80 MON,MON=MON.ASM           ; -> MON.REL + MON.PRN   (objfile,lstfile=srcfile)
L80 /P:F000,MON,MON/N/X/E     ; -> MON.HEX
W MON.HEX                     ; CP/M -> host  (.HEX and .PRN are text by extension)
W MON.PRN
```

The L80 switches, in the order they must appear:

Switch	Meaning
`/P:<addr>`	Program origin. **Must precede the module** — it sets the origin for the *next* module loaded.
`<module>`	Load `<module>.REL`.
`/N`	Name the output file after the preceding module (so `MON` → `MON.HEX`).
`/X`	Emit Intel **HEX** (not a `.COM`).
`/E`	Exit, writing the file.

For a ROM origin above the loader (`F000`+), L80 asks `Origin above loader memory, move anyway (Y or N)?` — answer `Y`. It builds in low memory but writes the `.HEX` at the true logical origin. (Verified byte-for-byte against the alternate route below.)

Alternate route: DRI `ASM.COM`

For a single absolute file, Digital Research's `ASM.COM` writes the `.HEX` **directly, no linker**: keep the absolute `ORG`, strip the M80-only `.8080` / `ASEG`, then `ASM FILE` → `FILE.HEX` + `FILE.PRN`. No memory limit, no prompt. It is a good cross-check — its output matched M80/L80 byte-for-byte on the SSM

monitor — but note **`ASM.COM` botches negative `DB` values** (`DB -1` assembles to `0C`, not `FF`), so where a source uses them, **M80/L80 is authoritative.**

Several modules into one part

A ROM built from independent modules at different origins (the SSM 8080 monitor is `F000`/`F600`/`F700`) is **not linked** — each module assembles on its own to its own `.HEX`, and you concatenate the images into one file. Keep **only the combined `.HEX` in `roms/<NAME>/`**: `cmake/embed_roms.cmake` globs `*.HEX` and takes the first, so a directory with several would embed the wrong one. Ship the module `.ASM`/`.PRN` for provenance, but one image.

Then wire it in

1. Drop the combined `.HEX` (and the `.ASM`/`.PRN` sources) in `roms/<NAME>/`, with a one-line `DESC`. The directory becomes a built-in ROM automatically.
2. Add its CRC32 row to `docs/roms.md` and a case to `tests/test_roms.cpp` — a built-in ROM is a hardware fact, so a mangled embed must fail the **build**, not a user. The test CRC is over `Image::flat()`, i.e. the `FF`-filled span `lo`–`hi`, so a part with gaps is `contiguous == false`.
3. Prove it runs. Mount it and boot: a monitor should reach its banner and respond to a command over the console.

A worked example, end to end, is `roms/SSM-8080MON/` and its `README.md`.

Review comments

You can leave notes on a document for Claude to act on, without those notes ever touching the tree. Mark up a *copy* of a file, ask Claude to address the comments, and the master `.md` in the repository is revised in place. The copy is scratch; the master never carries a marker, so there is nothing to strip out afterward and nothing that can leak into a build.

The mechanism is an HTML comment with a sentinel. HTML comments are invisible in rendered Markdown and in the built PDFs, and they are trivial to `grep` for — the same trick the generated `docs/manual/ref/*.md` banner and the `docs/boards/_TEMPLATE.md` author notes already use.

The marker

Any HTML comment that begins with `@claude`:

```
Some sentence that reads awkwardly. <!-- @claude: too terse — expand this. -->
```

For a longer note, use the block form:

```
<!-- @claude
This whole section assumes the reader already knows what a hard-sector disk is.
Add a one-paragraph primer before it.
-->
```

When several people are annotating one file, tag who wrote the note:

```
<!-- @claude(patrick): reword this for a first-time user. -->
```

Place the marker next to the text it is about — the sentence it follows, or immediately above the section it refers to. Claude anchors the change to where the marker sits, so position is how you say *what* the comment is about.

The workflow

1. **Copy** the file you want to review to somewhere outside the repository, keeping its filename. `docs/devguide/theory.md` becomes `~/Desktop/theory.md`, say. The basename is the link back to the master, so do not rename it.
2. **Annotate** the copy: drop `@claude` markers wherever you want a change.
3. **Ask** Claude to act on it — "*address the @claude comments in `~/Desktop/theory.md`*". Claude reads the markers, matches the copy to the repo file of the same name, and revises the **master** to address

each one.

To see every outstanding marker across the tree (they should only ever appear in copies, never here):

```
grep -rn '<!-- @claude' .
```

What Claude does with them

These are the rules that keep the process safe — worth knowing so you can predict the result:

- **It matches by filename.** The annotated copy shares the master's basename; Claude finds the repo file with that name and edits *that*. If more than one file in the tree has the name (a bare `README.md`, an `ORDER`), Claude asks which one rather than guessing.
- **The copy carries comments, not content.** Your copy may be hours or days out of date. Claude treats the markers — and the text immediately around each one — as the input, and applies them to the *current* master. It does not fold unrelated edits from the copy back into the tree, so a stale copy cannot silently revert someone else's work.
- **It works on a branch,** per the project rule that every change lands on a branch off `master`.
- **There is nothing to clean up.** The master never had a marker, so none is left behind. Claude reports what it changed, comment by comment, and leaves your annotated copy alone — it is yours.

Annotating in place (the exception)

You *can* drop markers straight into a repo file instead of a copy — handy for a quick note you will resolve in the same sitting. If you do, Claude removes each marker once it is addressed, so none survives the commit.

One place this is not allowed: the **User Manual** (`docs/manual/*.md`). Its raw text — comments included — is grep-checked by `tests/acceptance/docs-manual.cmake` for anything that points outside the shipped package (`src/`, `tests/`, `ctest`, a header name, and so on). A marker that mentions one of those tokens will fail the build even though it never renders. For the manual, review a copy.